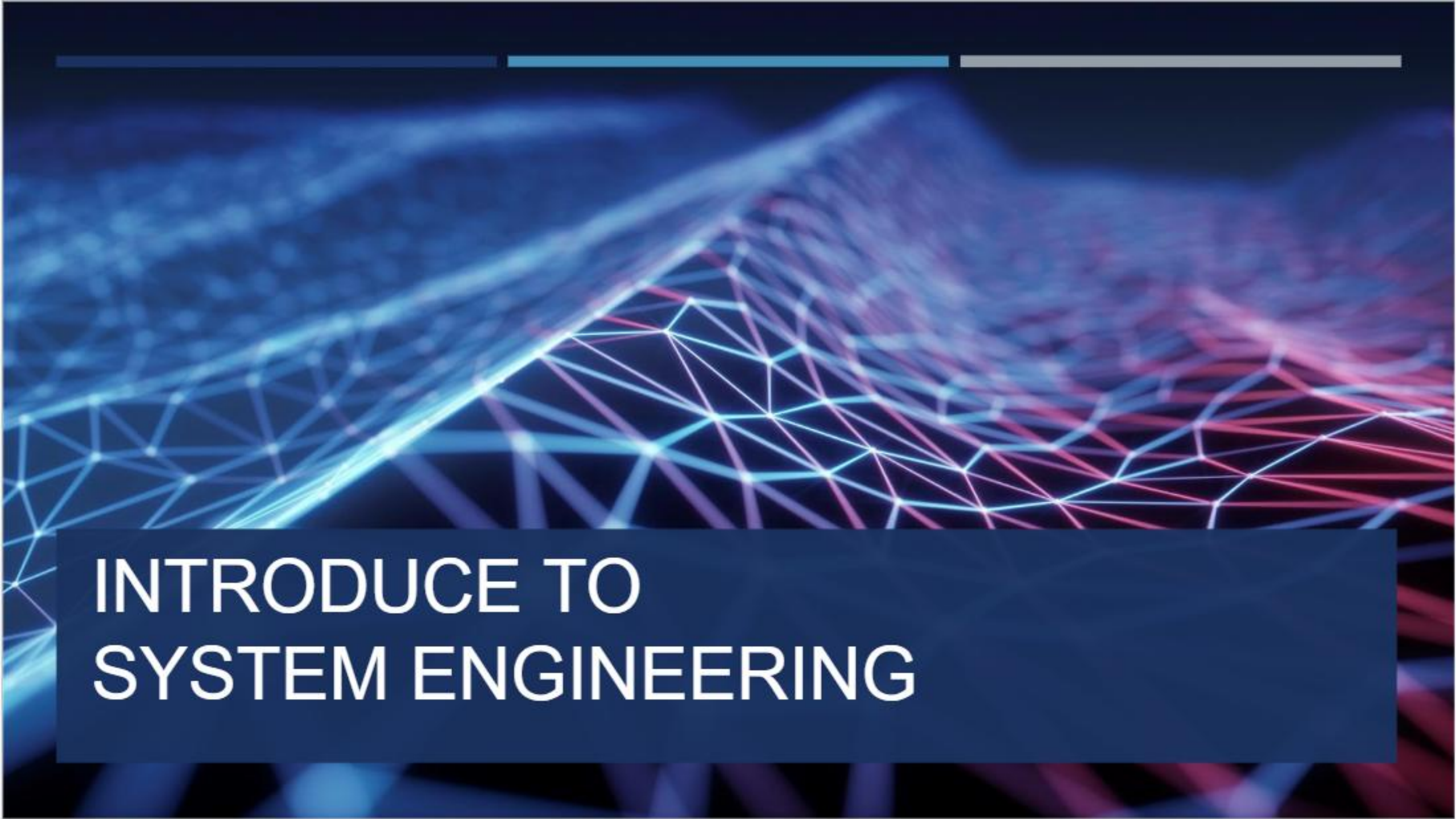




INTRODUCE TO SYSTEM ENGINEERING

CONTENT

1. What is a System?
2. What is Systems Thinking
3. What is Systems Engineering
4. Types of Systems Engineering
5. Systems engineering focuses on
6. Systems engineering process
7. The scope of systems engineering activities
8. Systems development life cycle

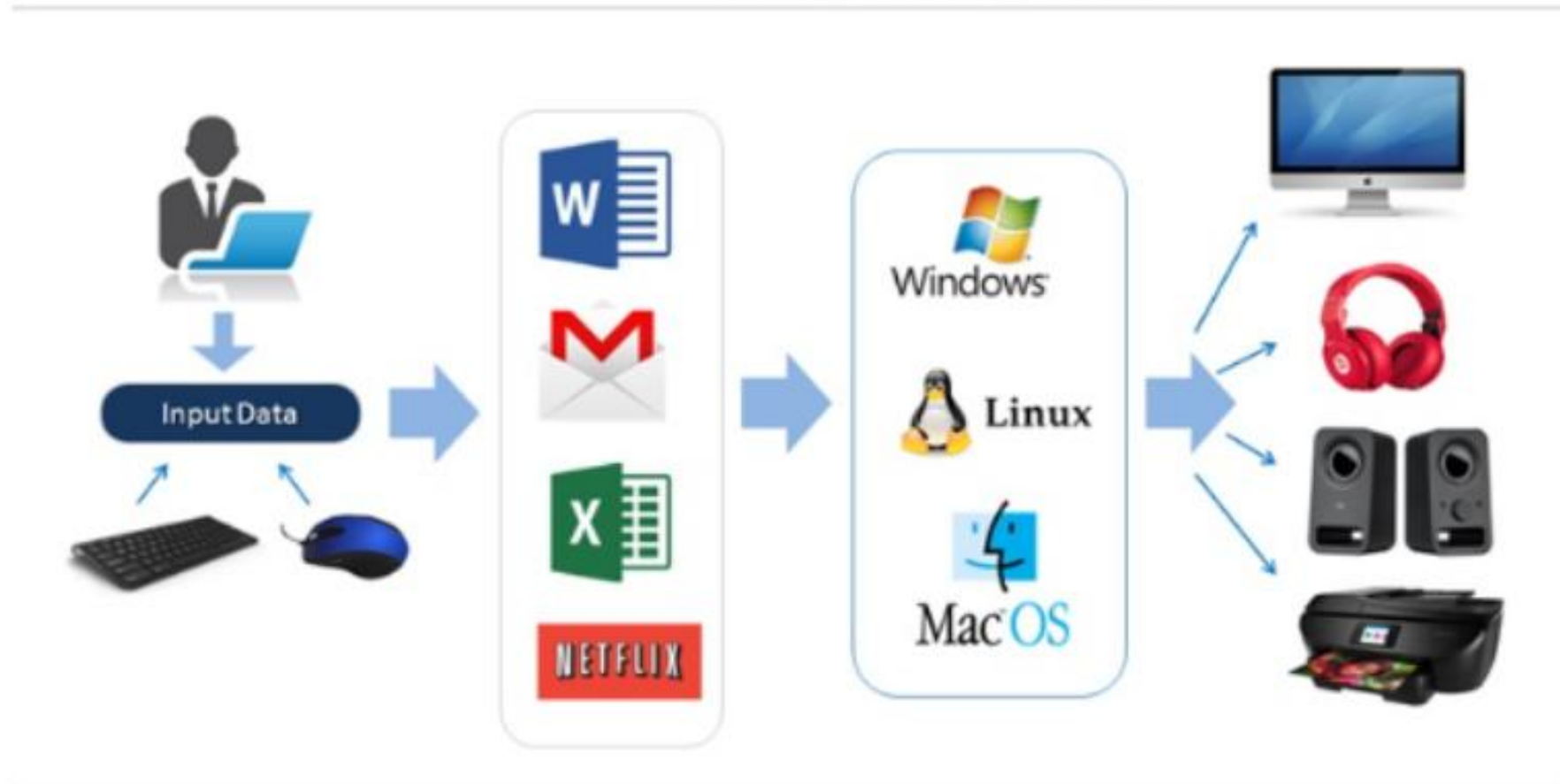
WHAT IS A SYSTEM?



- A **system** is a group of interacting or interrelated entities that form a unified whole. A system, surrounded and influenced by its environment, is described by its boundaries, structure and purpose and expressed in its functioning. *(From Wikipedia)*
- The modern world has numerous kinds of systems that influence daily life. Some examples include transport systems; solar systems; telephone systems; weapons systems; ecological systems; space systems; etc

Examples: Computer System

Introduction To Computer System



Example: School System



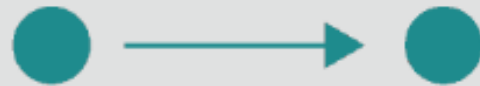
WHAT IS A SYSTEM?

In information and computer science:

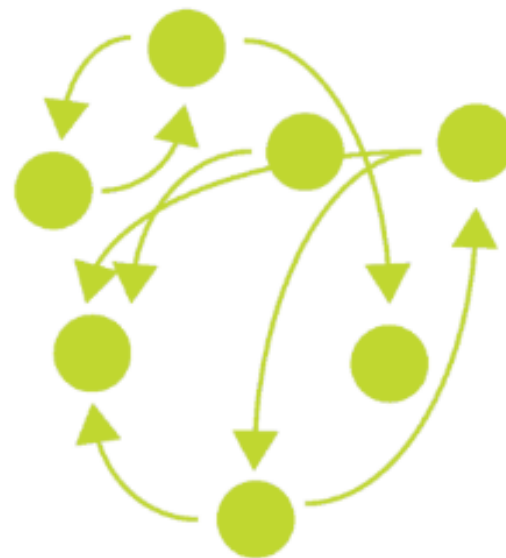
- **System** is a hardware system, software system, or combination, which has components as its structure and observable inter-process communications as its behavior.
- There are systems of counting, various library systems, etc..
- System can also refer to a framework, aka platform, be it software or hardware, designed to allow software programs to run.

WHAT IS SYSTEMS THINKING

Traditional thinking

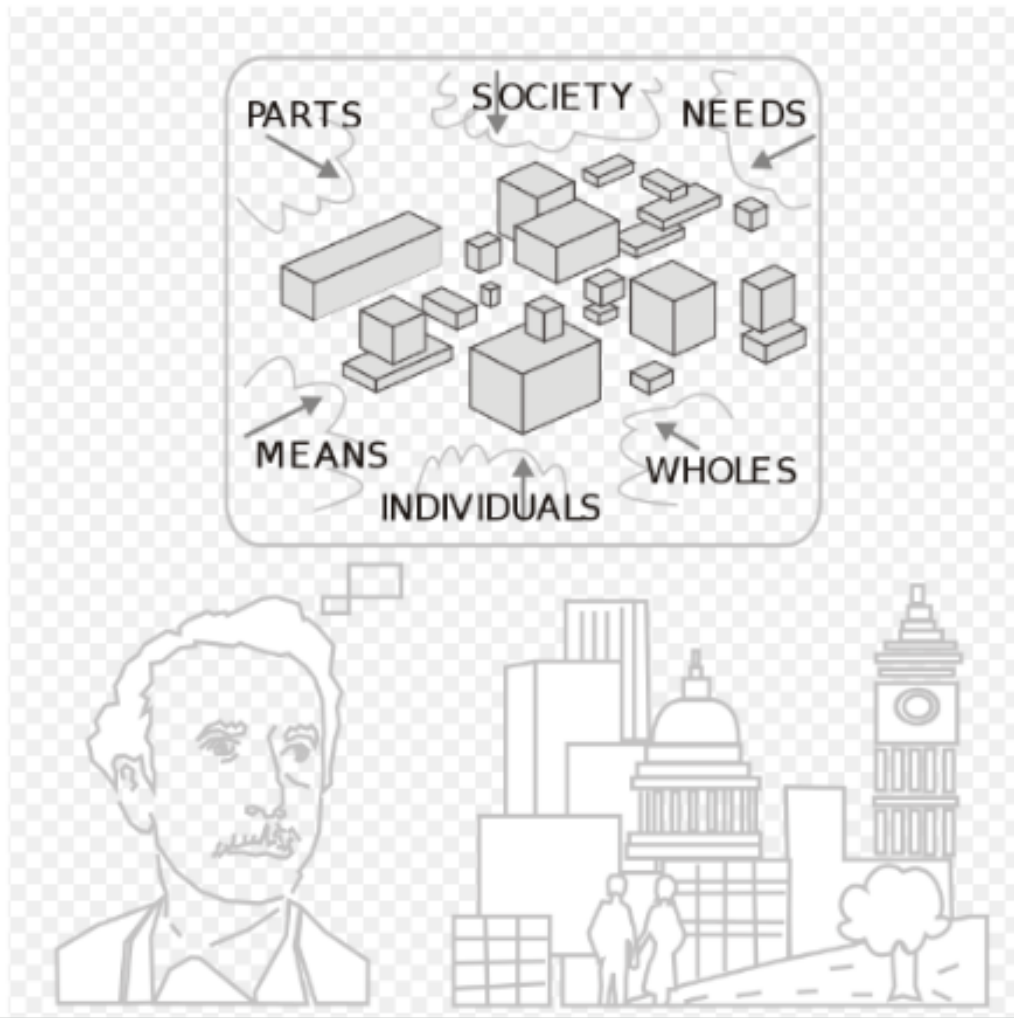


Systems thinking



KINDLING.xyz

WHAT IS SYSTEMS THINKING



Systems thinking is a way of making sense of the complexity of the world by looking at it **in terms of wholes** and relationships rather than by splitting it down into its parts.

From Wikipedia, the free encyclopedia

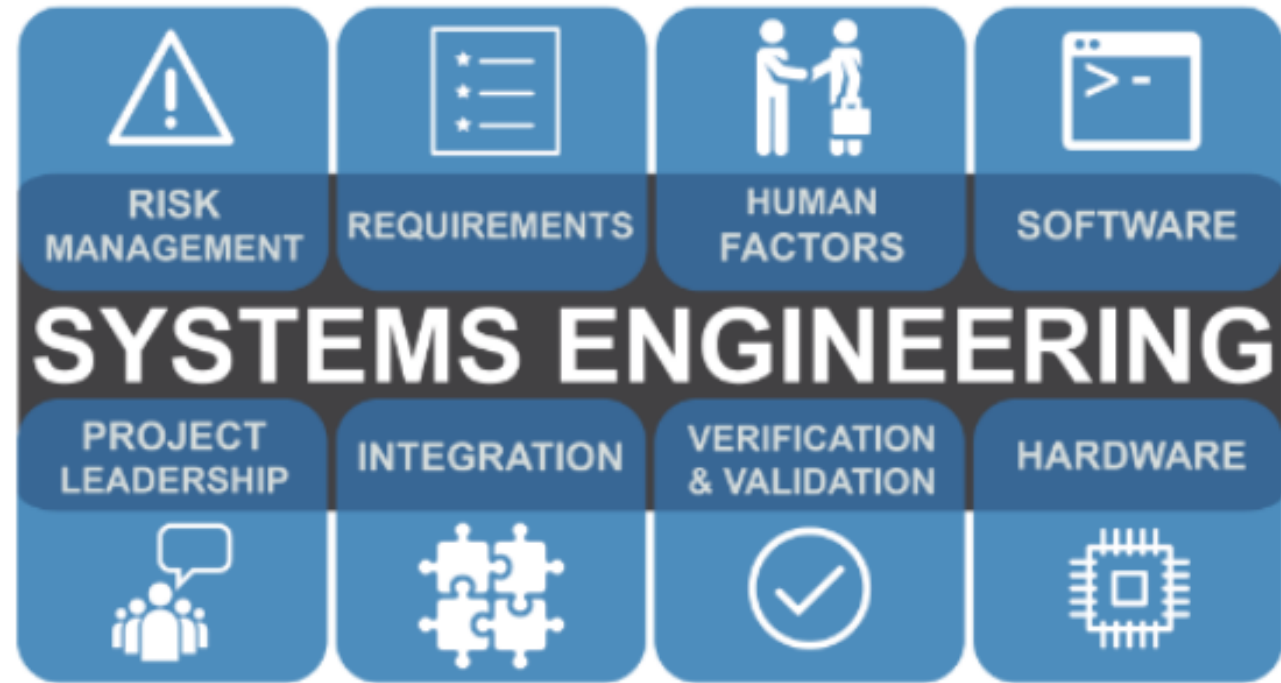
Q&A

Why systems thinking is important at work?

The Importance of Systems Thinking in Engineering

- Systems thinking is a mindset and strategy for solving problems. When someone uses systems thinking they analyze HOW something is happening and make changes to the elements that produce that result.
- In engineering, systems thinking is important because it helps engineers design systems that are efficient, effective "Exploring the Unique Approaches of Systems Science Systems Engineering and Systems Thinking | LinkedIn" , and sustainable. By using systems thinking, engineers can identify the key components of a system and understand how they interact with one another. This allows them to design systems that are optimized to meet the needs of the user and the environment.

WHAT IS SYSTEMS ENGINEERING



It is an interdisciplinary field of engineering and engineering management that focuses on how to design and manage complex systems over their life cycles. At its core, systems engineering utilizes systems thinking principles to organize this body of knowledge. The individual outcome of such efforts, an engineered system, can be defined as a combination of components that work in synergy to collectively perform a useful function. *(From Wikipedia)*

WHAT IS SYSTEMS ENGINEERING



Optimization methods



Systems engineering deals with work-processes, optimization methods, and risk management tools in such projects.

WHAT IS SYSTEMS ENGINEERING

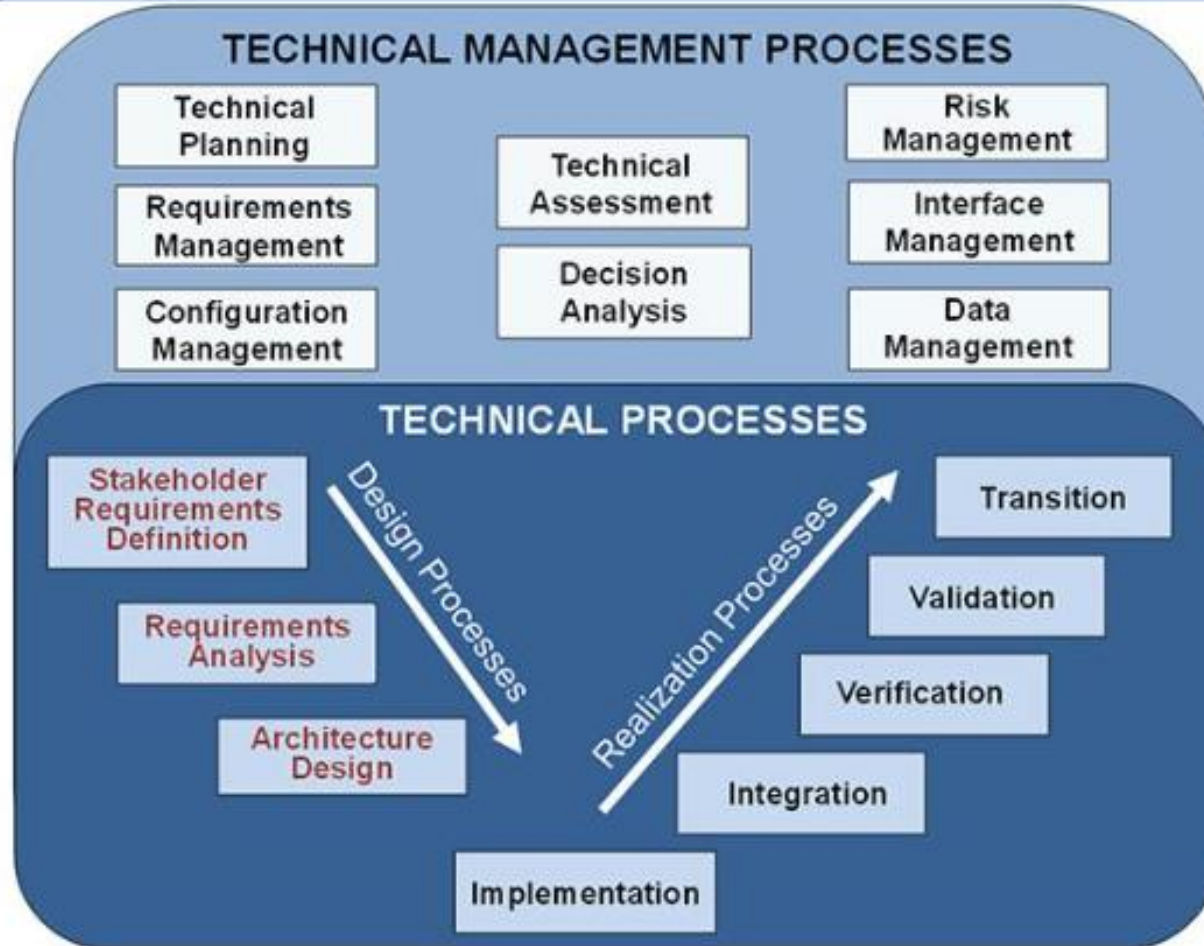


It overlaps technical and human-centered disciplines such as industrial engineering, mechanical engineering, manufacturing engineering, control engineering, software engineering, electrical engineering, cybernetics, organizational studies, civil engineering and project management

TYPES OF SYSTEMS ENGINEERING

- 1. Product Systems Engineering** is the traditional systems engineering focused on the design of physical systems consisting of hardware and software
- 2. Enterprise Systems Engineering** is the application of systems engineering principles, concepts, and methods to the planning, design, improvement, and operation of an enterprise.
- 3. Service Systems Engineering** has to do with the engineering of service systems.

SYSTEMS ENGINEERING PROCESS



The systems engineering process can be decomposed into:

1. *A Systems Engineering Technical Process,*
2. *A Systems Engineering Management Process.*

MANAGING COMPLEXITY

Systems engineering encourages the use of tools and methods to better comprehend and manage complexity in systems such as:

- *System architecture,*
- *System model, Modeling, and Simulation,*
- *Optimization,*
- *System dynamics,*
- *Systems analysis,*
- *Statistical analysis,*
- *Reliability analysis, and*
- *Decision making*

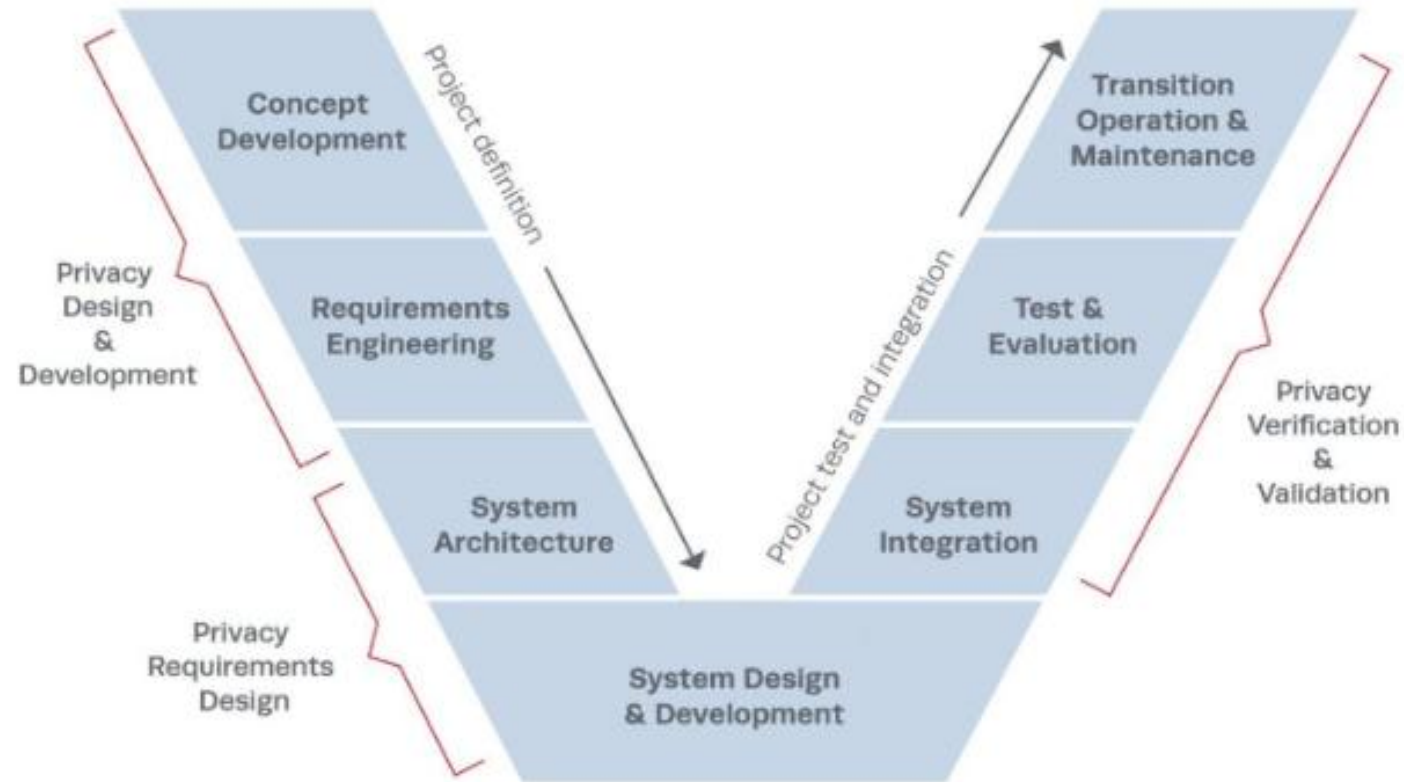


SYSTEMS DEVELOPMENT LIFE CYCLE



Systems development life cycle (SDLC), also referred to as the **application development life-cycle**, is a process for planning, creating, testing, and deploying.

SYSTEMS DEVELOPMENT LIFE CYCLE



Systems Engineering Process (www.mitre.org)



INTRODUCE TO SYSTEM DEVELOPMENT LIFE CYCLE

CONTENT

1. Systems development life cycle overview
2. 9 common steps in SDLC
3. Systems development life cycle Models

SYSTEMS DEVELOPMENT LIFE CYCLE

- **WHAT?**

A systems development life cycle is composed of a number of clearly defined and distinct work phases.

- **WHO?**

Systems engineers and systems developers

- **WHERE?**

The SDLC concept applies to a range of hardware and software configurations, as a system can be composed of hardware only, software only, or a combination of both.

SYSTEMS DEVELOPMENT LIFE CYCLE

■ Crucial factors

- In SDLC, documentation is crucial, regardless of the type of model chosen for any application, and is usually done in parallel with the development process.
- The most crucial factor for the success of a project may be how closely the particular plan was followed.

■ Steps in SDLC

- SDLC can be made up of multiple steps but there is no concrete set number of steps involved.
- There can be anywhere from 5-12 and the more

9 COMMON STEPS IN SDLC



Other steps which may appear include project initiation, functional specifications, detailed specifications, evaluation, end-of-life and other steps that can be created by splitting previous steps apart further.

ANALYSIS



- The existing system is evaluated.
- Deficiencies are identified.
- This can be done by interviewing users of the system and consulting with support personnel.

PLANNING



- The new system requirements are defined.
- In particular, the deficiencies in the existing system must be addressed with specific proposals for improvement.
- Other factors defined include needed features, functions and capabilities.

DESIGN



- The proposed system is designed.
- Plans are laid out concerning the physical construction, hardware, operating systems, programming, communications and security issues.

DEVELOPMENT



- The new system is developed.
- The new components and programs must be obtained and installed.
- Users of the system must be trained in its use.

TESTING



All aspects of performance must be tested. If necessary, adjustments must be made at this stage. Tests performed by quality assurance (QA) teams may include systems integration and system testing.

DEPLOYMENT



- The system is incorporated in a production environment.
- This can be done in various ways.
- The new system can be phased in, according to application or location, and the old system gradually replaced.
- In some cases, it may be more cost-effective to shut down the old system and implement the new system all at once

MAINTENANCE



- This step involves changing and updating the system once it is in place.
- Hardware or software may need to be upgraded, replaced or changed in some way to better fit the needs of the end-users continuously.
- Users of the system should be kept up-to-date concerning the latest modifications and procedures.

EVALUATION



- Looks at the overall project and considers how things went.
- Some of the questions that need to be answered include if the newly implemented system meets the initial business requirements and objectives, if the system is reliable and fault-tolerant, and if it functions according to the approved functional requirements.

DISPOSAL



In this phase, plans are developed for discontinuing the use of system information, hardware, and software and making the transition to a new system.

The purpose here is to properly move, archive, discard, or destroy information, hardware, and software that is being replaced, in a manner that prevents any possibility of unauthorized disclosure of sensitive data.

The disposal activities ensure proper migration to a new system. Particular emphasis is given to proper preservation and archiving of data processed by the previous system. All of this should be done in accordance with the organization's security requirements.



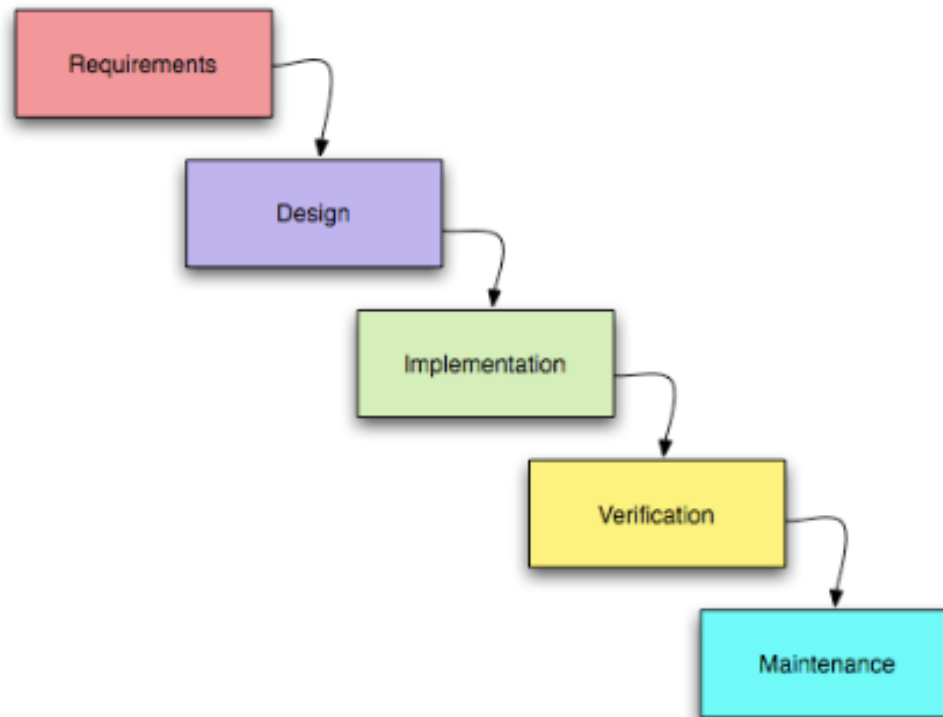
SYSTEMS DEVELOPMENT LIFE CYCLE MODELS

SYSTEMS DEVELOPMENT LIFE CYCLE MODELS

6 basic SDLC Models

- 1. Waterfall Model
- 2. V-Shaped Model
- 3. Iterative Model
- 4. Spiral Model
- 5. Big Bang Model
- 6. Agile Model
- Frequently, several models are combined into a hybrid methodology. Many of these models are shared with the development of software, such as waterfall or agile. Numerous model frameworks can be adapted to fit into the development of software.

WATERFALL MODEL



WATERFALL MODEL

Advantages

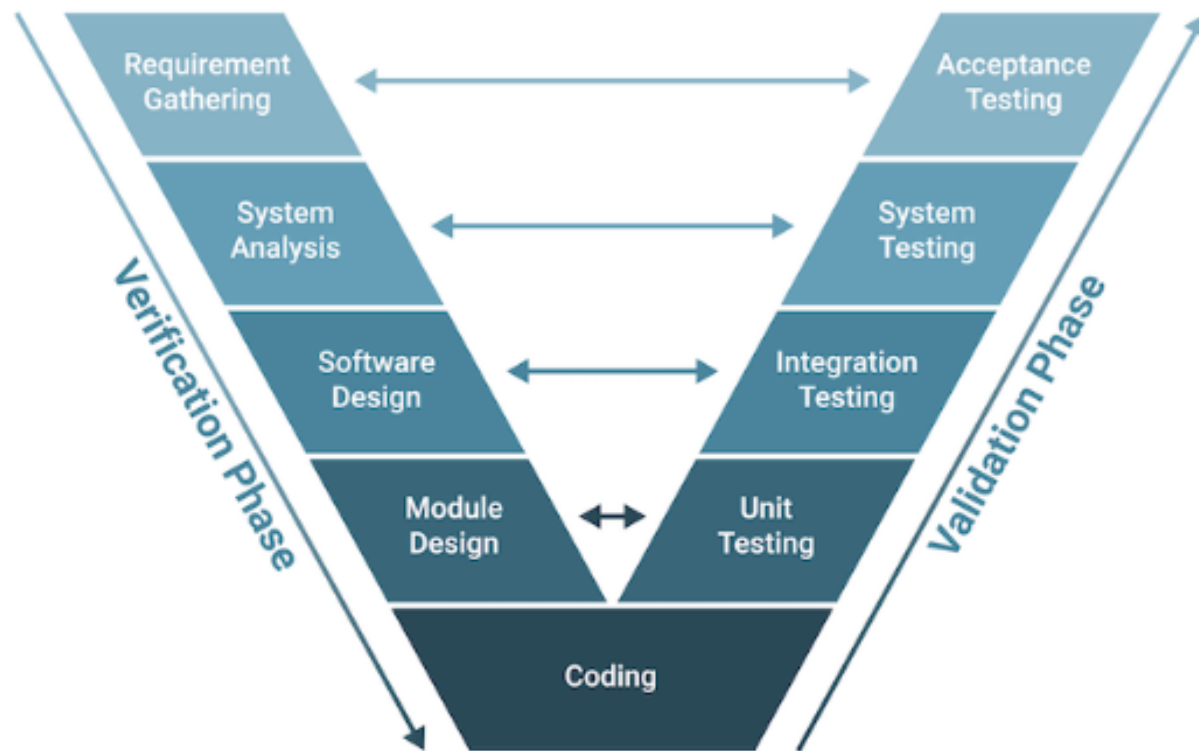
- Easy to Understand: simple and easy to understand.
- Individual Processing: phases - one at a time.
- Properly Defined: each stage in the model is clearly defined.
- Clear Milestones: very clear and well-understood milestones.
- Properly Documented: processes, actions, and results are very well documented.
- Reinforces Good Habits: good habits like define-before-design and design-before-code.
- Working: works well for smaller projects and projects where requirements are well understood.

WATERFALL MODEL

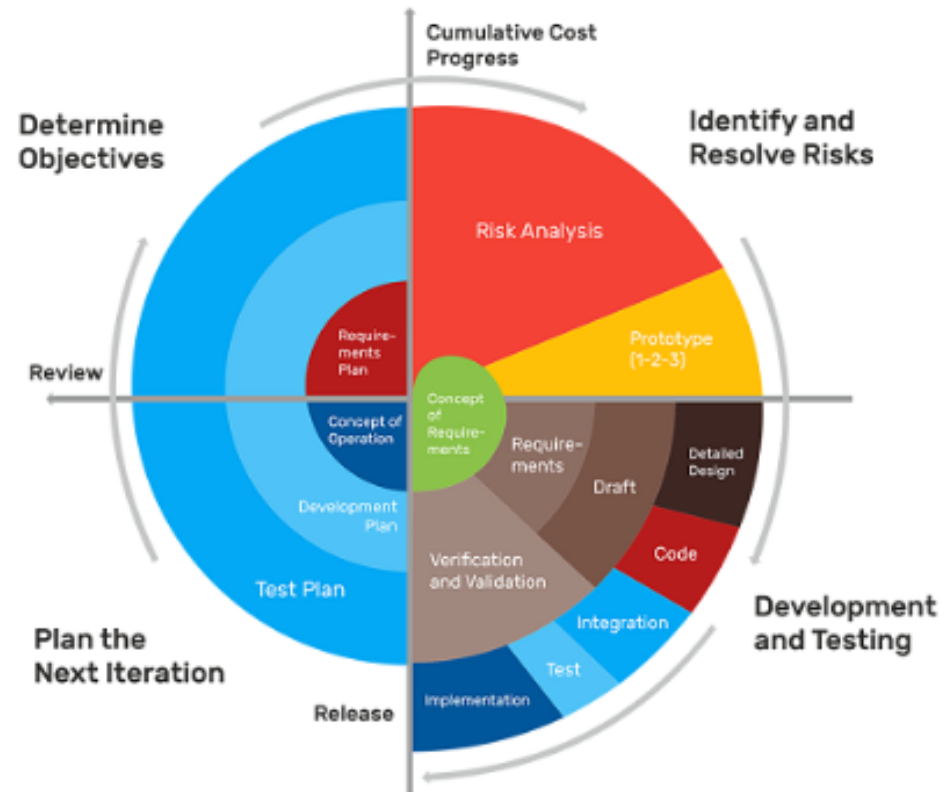
Disadvantages

- No Feedback Path
- Difficult to accommodate Change Requests:
- No Overlapping of Phases
- Limited Flexibility
- Limited Stakeholder Involvement
- Late Defect Detection
- Lengthy Development Cycle

V-MODEL

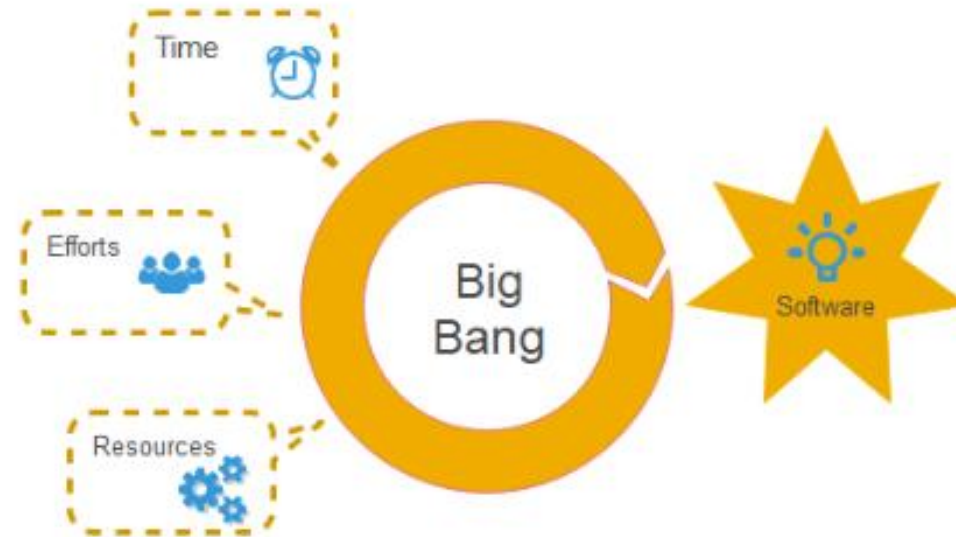


SPIRAL MODEL



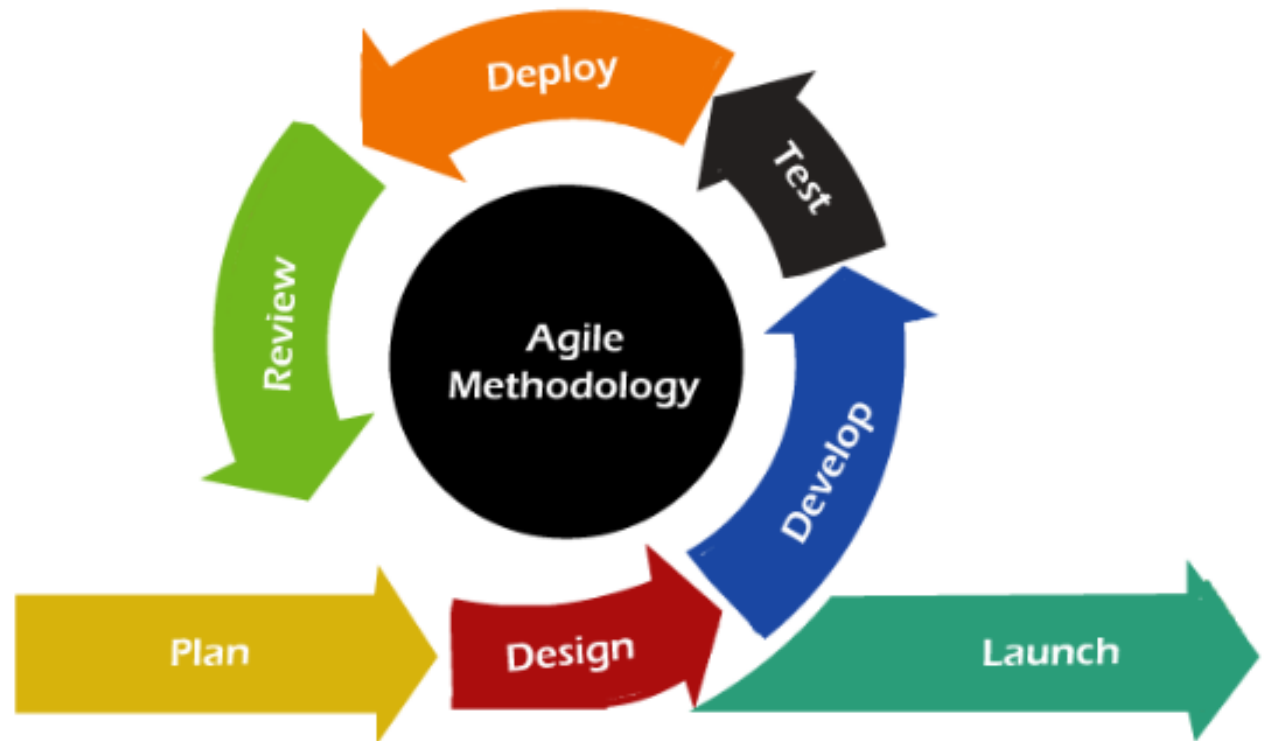
Spiral Model Methodology and its Phases

BIG BANG MODEL



AGILE MODEL

- Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- Agile Methods break the product into small incremental builds.
- These builds are provided in iterations. Each iteration typically lasts from about one to three weeks.
- Every iteration involves cross functional teams working simultaneously on various areas like: Planning, Requirements Analysis, Design, Coding, Unit Testing and Acceptance Testing.
- At the end of the iteration, a working product is displayed to the customer and important stakeholders.



STRENGTHS AND WEAKNESSES OF SDLC

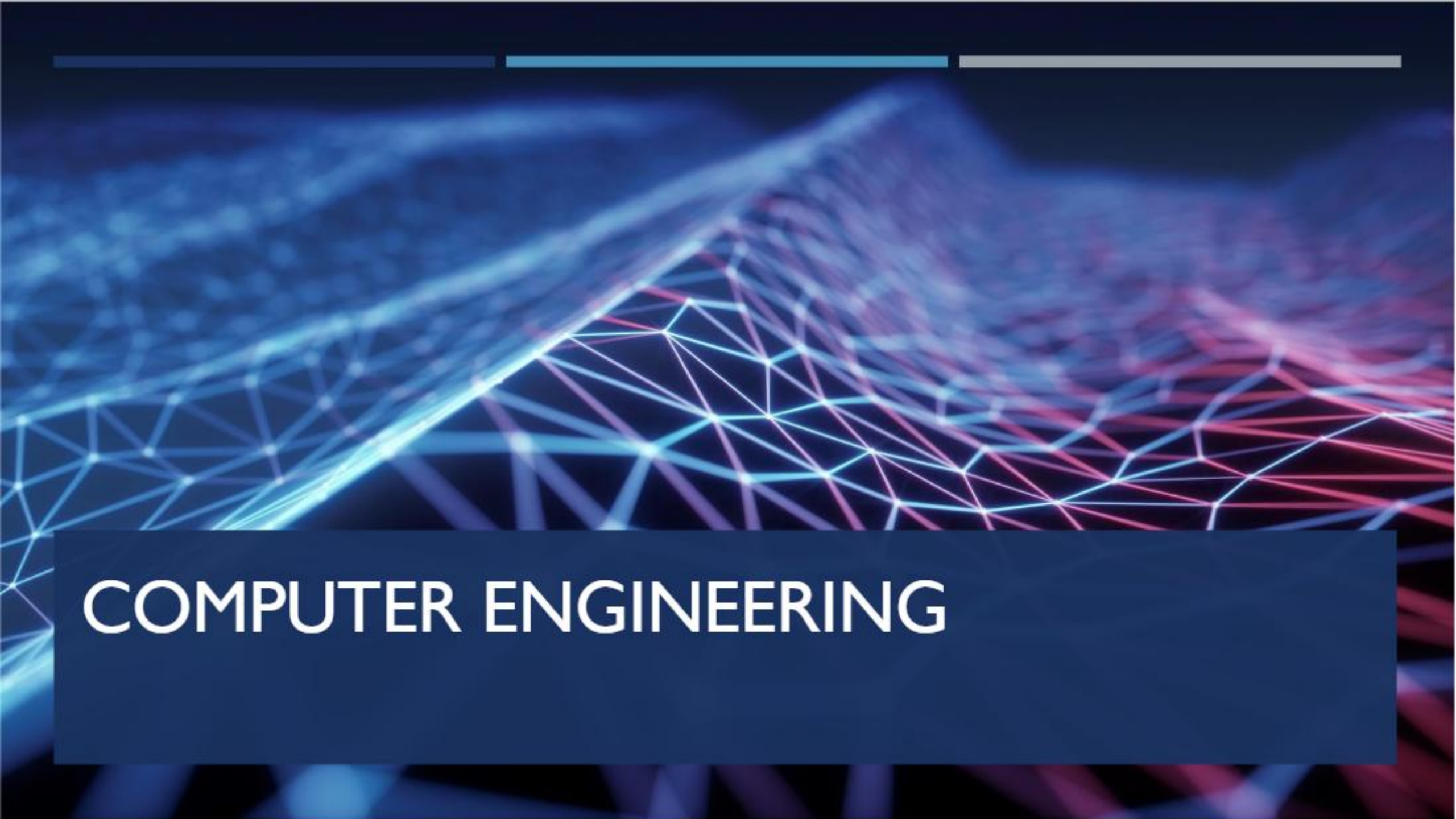
Advantages

- Is a very realistic approach to software development.
- Promotes teamwork and cross training.
- Functionality can be developed rapidly and demonstrated.
- Resource requirements are minimum.
- Suitable for fixed or changing requirements
- Delivers early partial working solutions.
- Good model for environments that change steadily.
- Minimal rules, documentation easily employed.
- Enables concurrent development and delivery within an overall planned context.
- Little or no planning required.
- Easy to manage.
- Gives flexibility to developers.

STRENGTHS AND WEAKNESSES OF SDLC

Disadvantages

- An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- There is a very high individual dependency, since there is minimum documentation generated.
- Transfer of technology to new team members may be quite challenging due to lack of documentation.

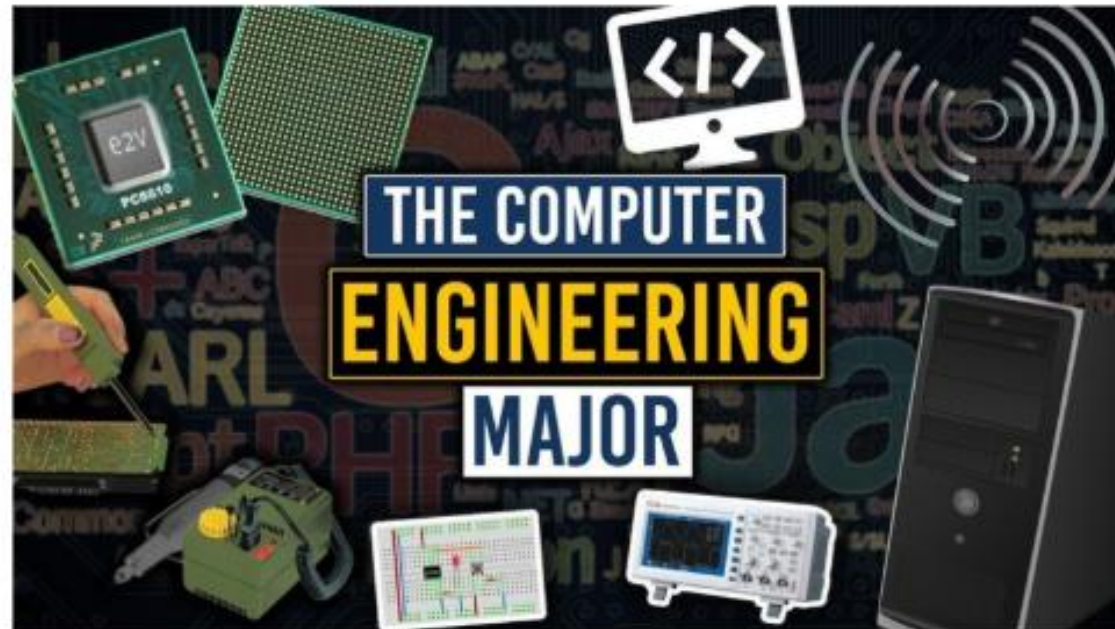


COMPUTER ENGINEERING

CONTENT

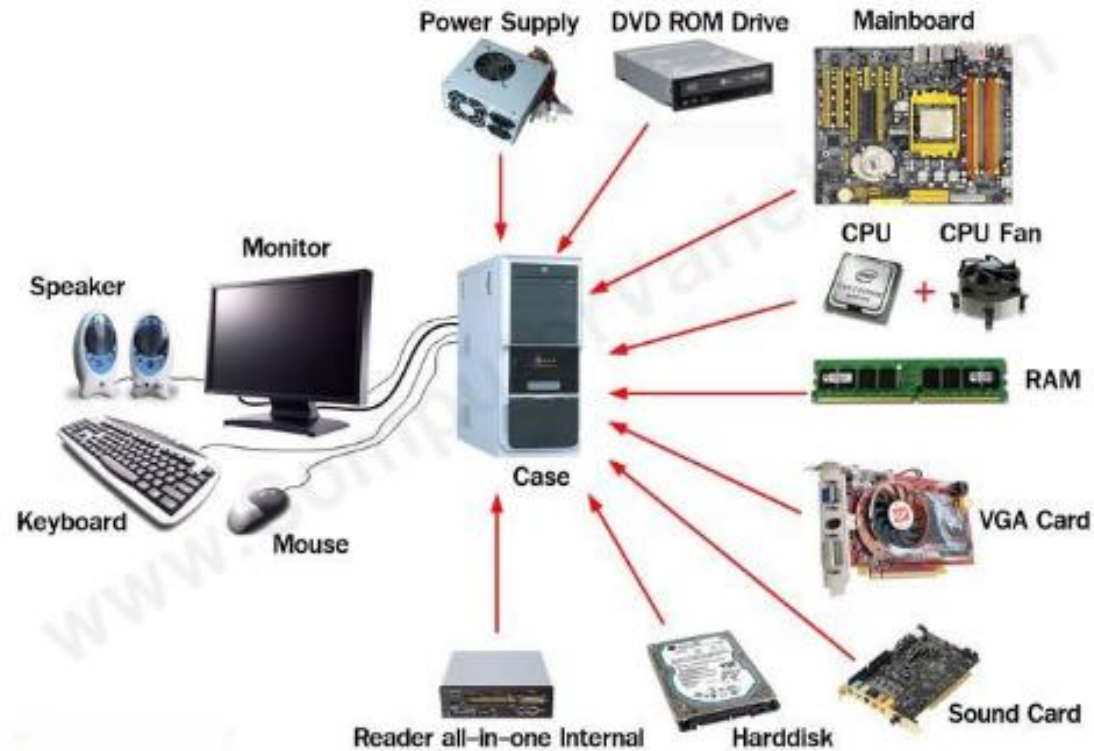
1. Computer engineering
2. Hardware
3. Software
4. Desktop Application
5. Web Application
6. Mobile Application

COMPUTER ENGINEERING



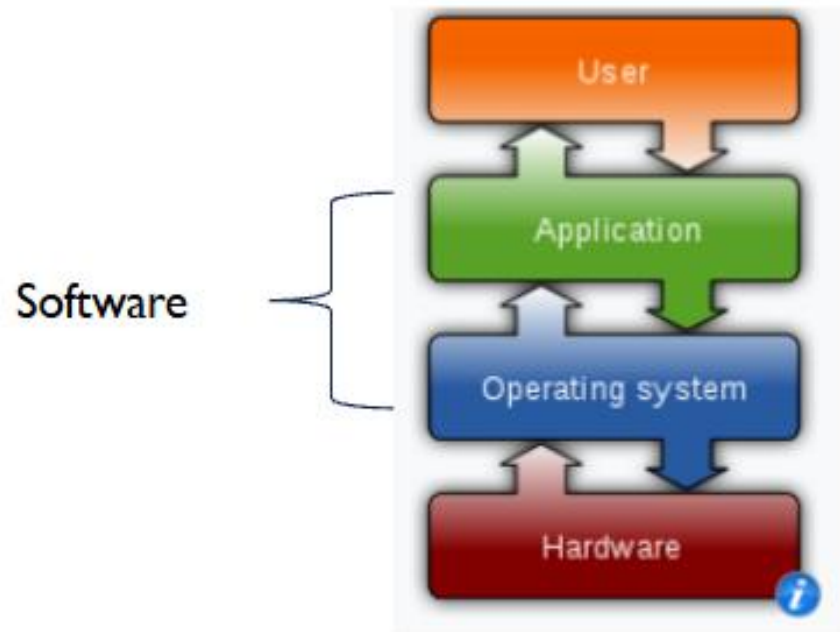
- **Computer engineering** (CpE) is a branch of engineering that integrates several fields of computer science and electronic engineering required to develop computer hardware and software.
- There are two major focuses in computer engineering: **hardware** and **software**. Hardware is so-termed because it is "hard" or rigid with respect to changes, whereas software is "soft" because it is easy to change.

HARDWARE



Computer hardware includes the physical parts of a computer, such as the case, central processing unit (CPU), monitor, mouse, keyboard, computer data storage, graphics card, sound card, speakers and motherboard.

SOFTWARE



Software is a set of instruction, data or programs used to operate computers and execute specific tasks . Opposite of hardware, which describe the physical aspects of a computer, software is a generic term used to refer to application, scripts, and program that run on device.

SOFTWARE

Types of Software



The two major types of computer software are:

- Application Software
- System Software

SYSTEM SOFTWARE

Operating system



macOS
Proprietary ...

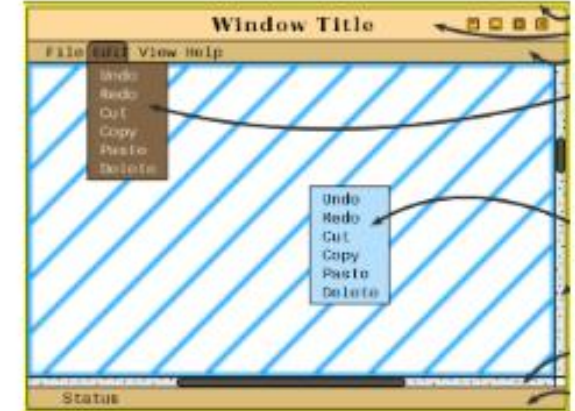


Windows 10
Shareware

Unity software



Shells and windowing systems



System software is a type of computer program designed to run a computer's hardware. It controls the operations of the computer hardware and provides an environment or platform for all the other types of software to work in. Examples of a system software include a operating system(OS), Device drivers, Middleware, Unity software, Shells and windowing systems.

APPLICATION SOFTWARE



Application software is a computer software package that performs a specific function for an end user or, in some instances, for another application. An application can be self-contained or a group of programs. Examples of an application include a word processor, a spreadsheet, an accounting application, a web browser, an email client, a media player, a file viewer, simulators, a console game or a photo editor.

APPLICATION SOFTWARE

Application Software Type	Examples
Word processing software: Tools that are used to create word sheets and type documents etc.	Microsoft Word, WordPad, AppleWorks and Notepad
Spreadsheet software: Software used to compute quantitative data.	Apple Numbers, Microsoft Excel and Quattro Pro
Database software: Used to store data and sort information.	Oracle, MS Access and FileMaker Pro
Application Suites: A collection of related programs sold as a package.	OpenOffice, Microsoft Office
Multimedia software: Tools used for a mixture of audio, video, image and text content.	Real Player, Media Player
Communication Software: Tools that connect systems and allow text, audio, and video-based communication.	MS NetMeeting, IRC, ICQ
Internet Browsers: Used to access and view websites.	Netscape Navigator, MS Internet Explorer, and Google Chrome
Email Programs: Software used for emailing.	Microsoft Outlook, Gmail, Apple Mail

PROGRAMMING SOFTWARE



Programming software are programs that are used to write, develop, test, and debug other software, including apps and system software. Examples of a programming software include a text editor, assembler/compiler, simulator, high – level language simulator, integrated development environments (IDEs).

DRIVER SOFTWARE



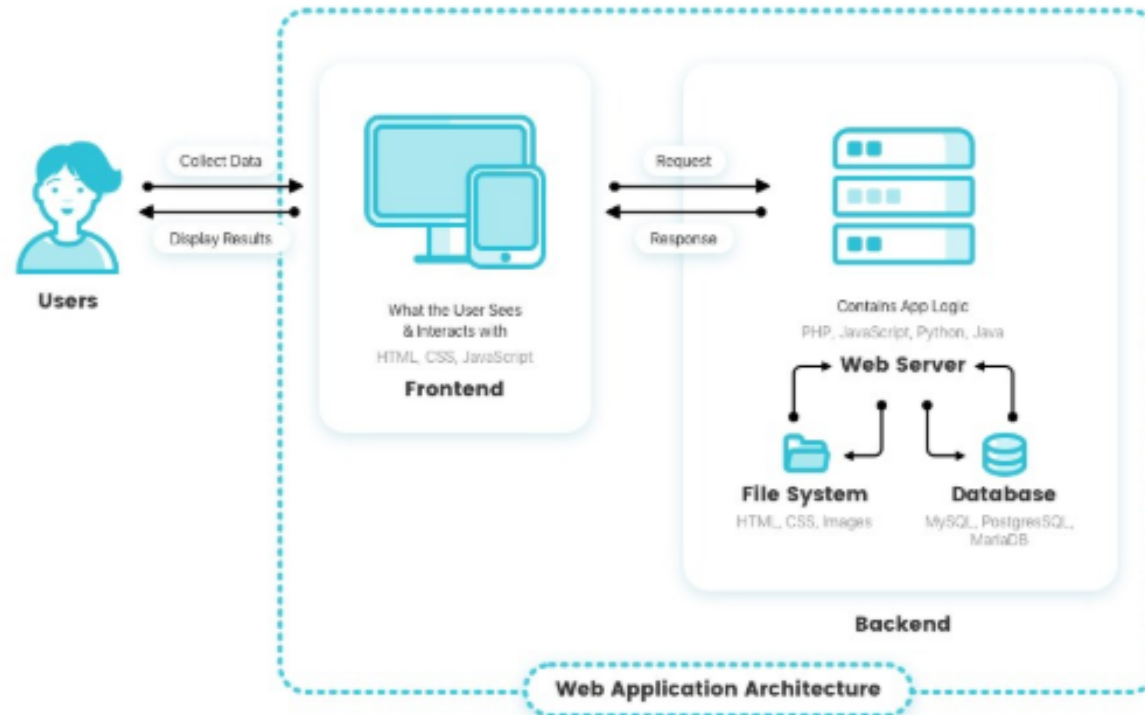
Driver software is often classified as one of the types of system software. They operate and control devices and peripherals plugged into a computer. They do this by translating commands of an Operating System for the Hardware or devices, assigning duties.

DESKTOP APPLICATIONS



A desktop application means any software that can be installed on a single computer (laptop or a desktop) and used to perform specific tasks.

WEB APPLICATION



- A **web application** (or **web app**) is application software that runs on a web server. Web applications are accessed by the user through a web browser with an active internet connection.
- Examples of commonly-used web applications include: web-mail, online retail sales, online banking,...



Analysis Phase

CONTENT

1. Analysis phase
2. Objective of the analysis phase
3. Overview of analysis phase
4. Input to the analysis phase
5. Main activities of the analysis phase
6. Output of the analysis phase
7. Best practices for an analysis phase

Analysis

Analysis: the detailed study or examination of something in order to understand more about it. (*Oxford English Dictionary*).

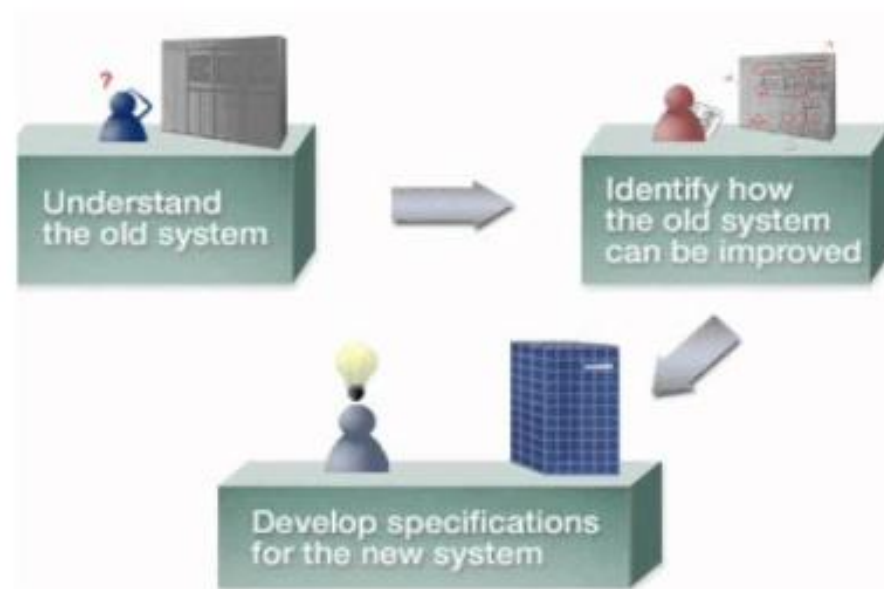
Analysis is the process of breaking a complex topic or substance into smaller parts in order to gain a better understanding of it.(*Wikipedia*).



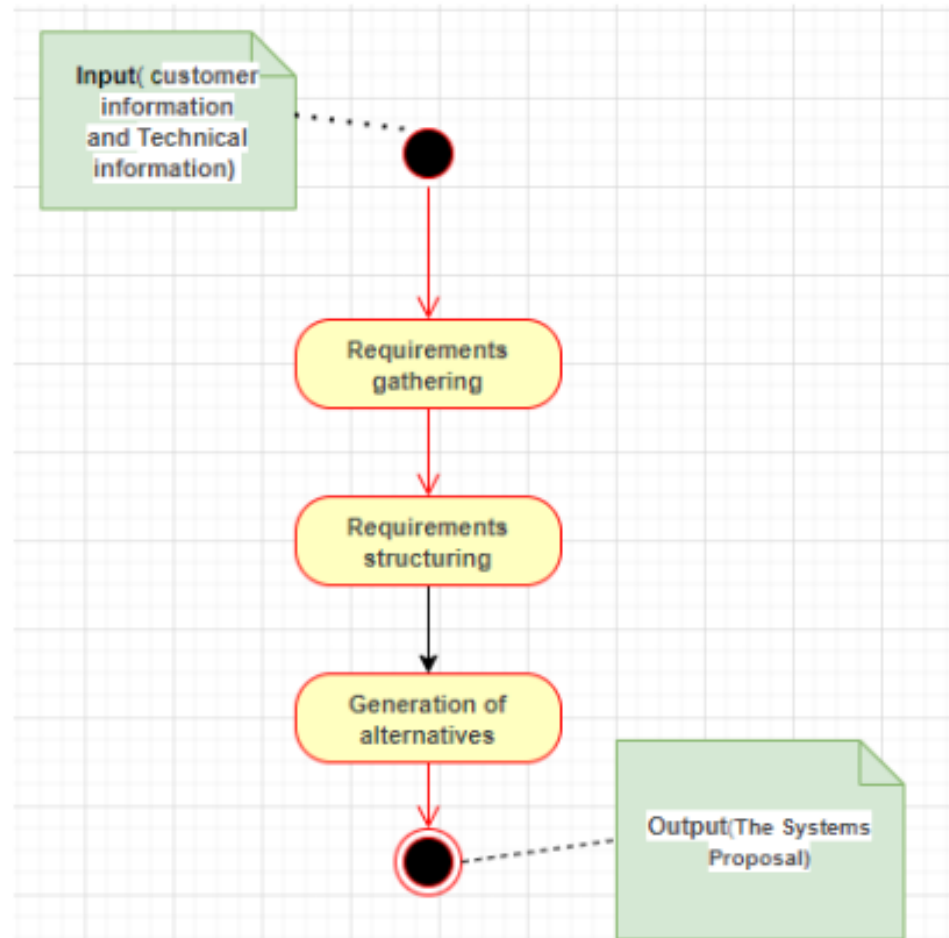
Objective of the analysis phase

To study the **current** system, organization, and problems to determine what changes need to be made in order to achieve the organization's goal.

The focus of the analysis phase is **current** situation, not the new system that will be implemented.



Overview of analysis phase



The activity diagram of analysis phase

Input to the analysis phase

1. Customer information

- from interviews, observations, documents, etc.
- information about goals, objectives, problems, work processes, current systems, constraints, etc.

2. Technical information

- from previous experience, web searches, vendor literature, previous education
- information about what technology is available, what it can do, how it works, and how much it costs



Main activities of the analysis phase

- **Requirements gathering:** using interviews, observations, documents, etc. to learn about the customer's business, their current systems, and its problems.
- **Requirements structuring:** using some kind of systematical and standard, well-structured methods such as data flow diagram, decision table, decision tree, entity-relationship diagram, use case etc. to model and organize the information gathered.
- **Generation of alternatives:** based on the requirements, investigate possible changes that would achieve the customer's objectives. It is the automatic result from the first two processes.

Output of the analysis phase

The Systems Proposal

- Summarizes all the information gathered and structured during analysis
- Presents alternative solutions
- Compares solutions in terms of costs, benefits, feasibility, ability to solve problems, etc.
- Provides all the information the customer needs to make a decision
- Recommends one solution

Best practices for an analysis phase

- Problem identification, definition and capture: The requirement analyst should identify the problem along with the business users and define it accurately. The requirement definition should be able to provide information on:
 - The problems the solution is aimed to solve
 - The benefits expected from the solution
- The feasibility of the requirements
- High Level Description of solution: The solution planned to be developed should be described at a high level along with the business needs it caters to.

Best practices for an analysis phase

- Address the needs of all the stakeholders and the users: This is a very important part of the requirement analysis and a step which needs to be meticulously followed before freezing the requirements. This would help in the deployment phase of the project too, by getting the users adaptable to the new process or application.
- Solution definition: The solution's dos and don'ts both should be captured in detail. This would help in clearing up any ambiguity whatsoever.
- Feature definition: The application's planned features need to be captured at length. The functional and non-functional requirements need to be captured in detail along with the details on how the project is going to be executed etc.



REQUIREMENT

Requirement

“ Requirement is a usable representation of a need”

A requirement as a

1. *condition or capability **needed** by a user to solve a problem or achieve an objective.*
2. *condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document.*
3. *documented representation of a condition or capability in (1) or (2).*

(The IEEE Glossary of Software Engineering Terminology and the Business Analysis Body of Knowledge® (BABOK®)).

Requirement

The BABOK® defines the following requirements types:

1. Business requirement
2. User (stakeholder) requirement
3. Functional (solution) requirement
4. Non-functional (quality of service) requirement
5. Implementation (transition).

Note that these terms are overloaded and often have different definitions within some organizations.

Business requirement

Description:

They are **statements of goals**, objectives, and outcomes that describe **why a change** has been initiated. They can **apply to the whole** of an enterprise, a business area, or a specific initiative.

Examples:

1. *Reduce incorrect processed orders by 50% by the end of next.*
2. *Increase repeat orders from customer by 10% within 6 months after deployment.*
3. *Implement an Organization-wide Inventory management system that will keep a track of the goods stored in the inventory, reduce wastage of material and increase the overall profits of the Organization by systematic administration.*

Business requirement

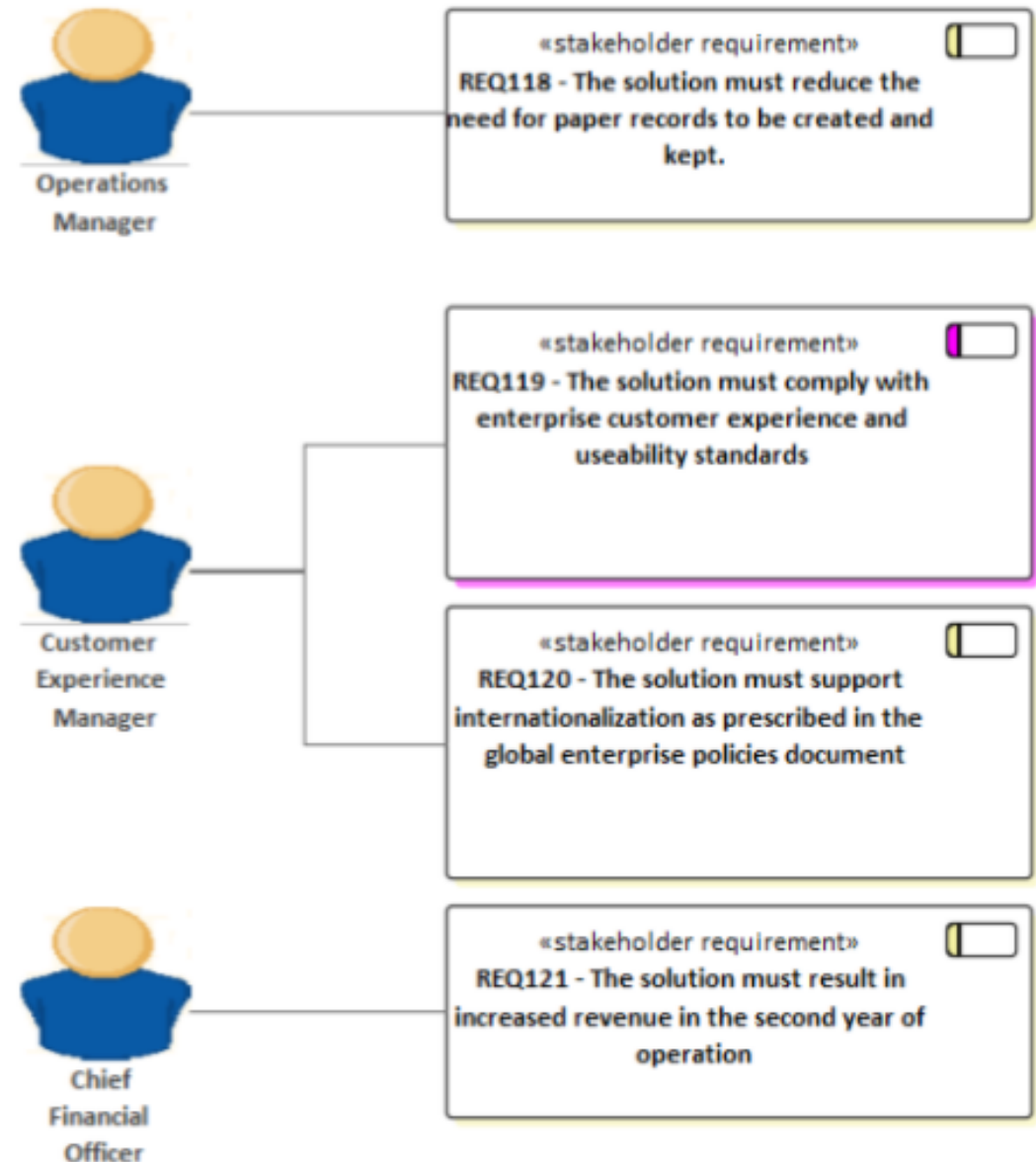
1. Defines the high-level needs of the organization and doesn't go into the details. Essentially, it answers 'What' the organization wants?
2. It states the benefits that will be attained when the requirement will be achieved. It answers 'Why' the organization wants it?
3. Is written from the perspective of the organization and not from the perspective of any stakeholder.

Practice: Make the Business Requirement for your project

Stakeholder requirement

They describe the **needs of stakeholders** that must be met in order to achieve the **business requirements**.

Practice: Make the Stakeholder Requirement for your project.

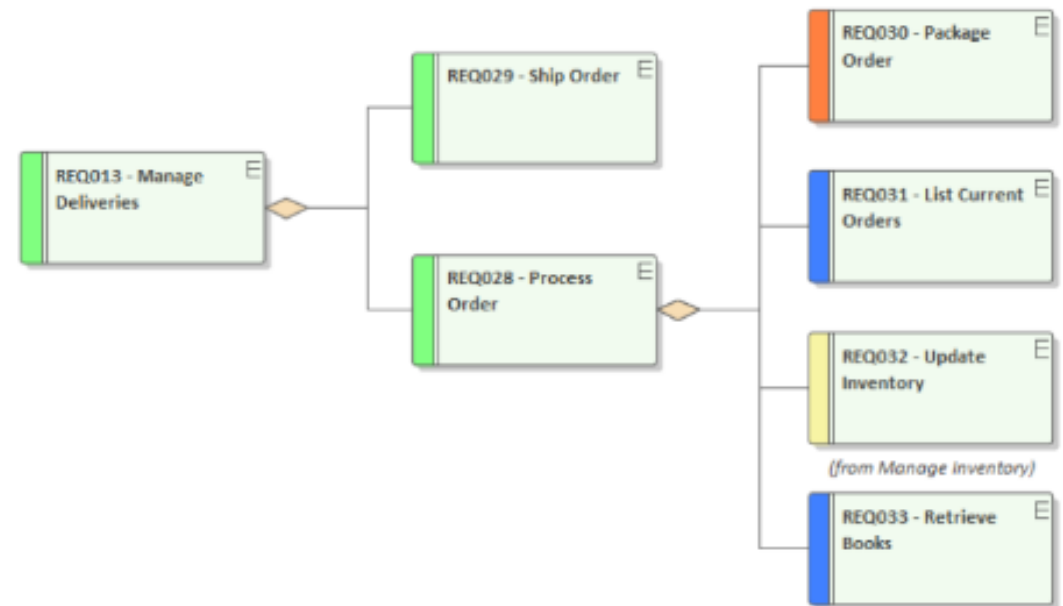


Functional (solution) requirement

They describe the capabilities that a solution must have in terms of the **behavior** and **information** that the solution will manage.

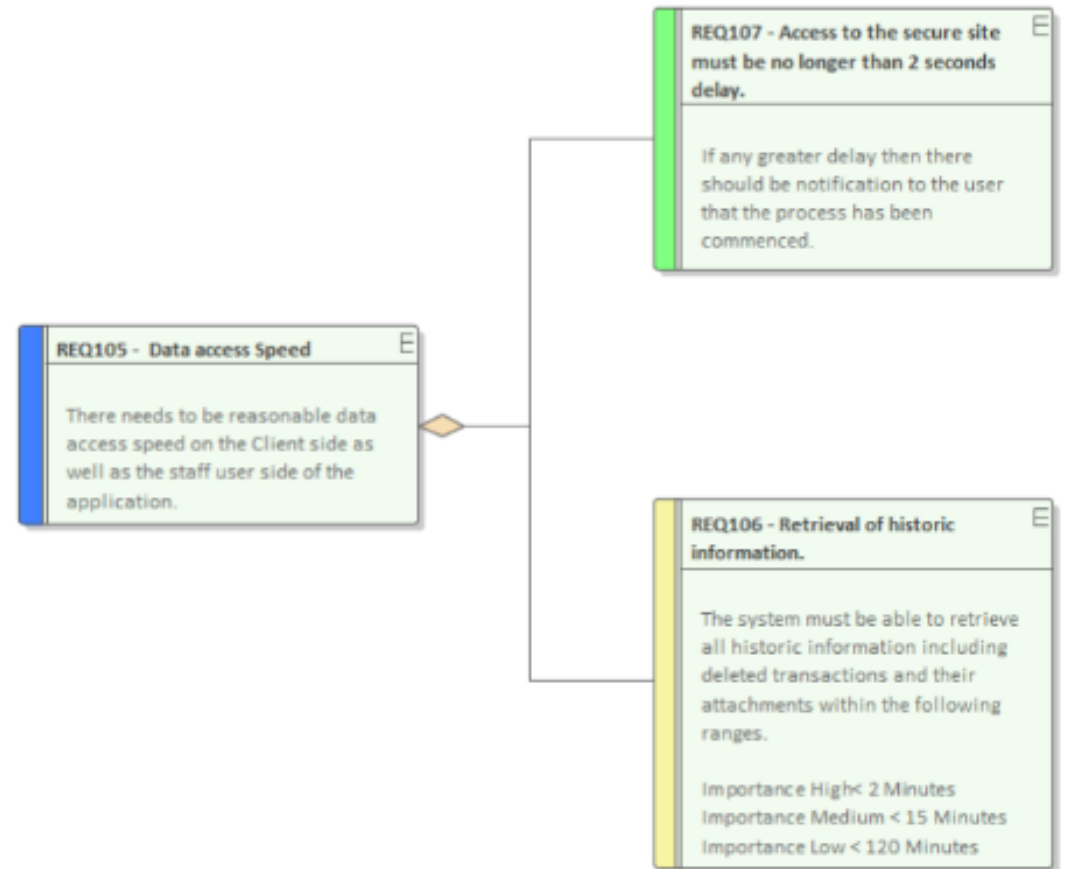
ID	Description
FR-1	Reader should be able to read the articles.
FR-2	Blog should be able to write an article.
FR-3	Blog should be able to edit a page.

Table of functional requirement)



Non-functional (quality of service) requirement

They describe the capabilities that the solution must have and the conditions the solution must meet **to facilitate transition** from the current state to the future state, but which are not needed once the change is complete.



Some requirement document

	Business Requirements Document (BRD)	Software Requirement Specifications (SRS)	Functional Requirement Specifications (FRS)
Other names		Product Requirements Document (PRD) and System Requirements Specification	Functional Specifications Document (FSD), Product Specification Document (PSD), Functional Specs (FS)
Created By	Business Analyst	Business/System Analyst	Business/System Analyst/Implementation Leads
Contains	High level business requirements and stakeholder requirements	Detailed functional requirements, non-functional requirements and use cases	Granular functional requirements, data flow and UML diagrams
Used By	Upper and middle management	Project managers, SMEs (subject matter experts), technical and implementation lead	Technical leads, development teams and testing teams.
Prepared in	Initiation phase	Planning phase	Planning phase
Answers	'Why' the requirements are being undertaken	'What' requirements must be fulfilled to satisfy business needs	'How' exactly the system is expected to function
Example	Improve efficiency by tracking the employee time in office	Proposed software will contain following modules: Login, Administrator, Employee and Reporting	Login module will contain fields like: Enter username, Enter password, Submit button



Unified Modeling Language

CONTENT

1. What is UML
2. Building Blocks
3. Architecture

What is UML

- UML (Unified Modeling Language) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.
- UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.
- OMG is continuously making efforts to create a truly industry standard.
- UML is not a programming language but tools can be used to generate code in various languages using UML diagrams. UML has a direct relation with object oriented analysis and design. After some standardization, UML has become an OMG standard.

Building blocks

UML is composed of three main building blocks, i.e., things, relationships, and diagrams. Building blocks generate one complete UML model diagram by rotating around several different blocks. It plays an essential role in developing UML diagrams. The basic UML building blocks are enlisted below:

1. Things
2. Relationships
3. Diagrams

Building blocks - Things

Things

Anything that is a real world entity or object is termed as things. It can be divided into several different categories:

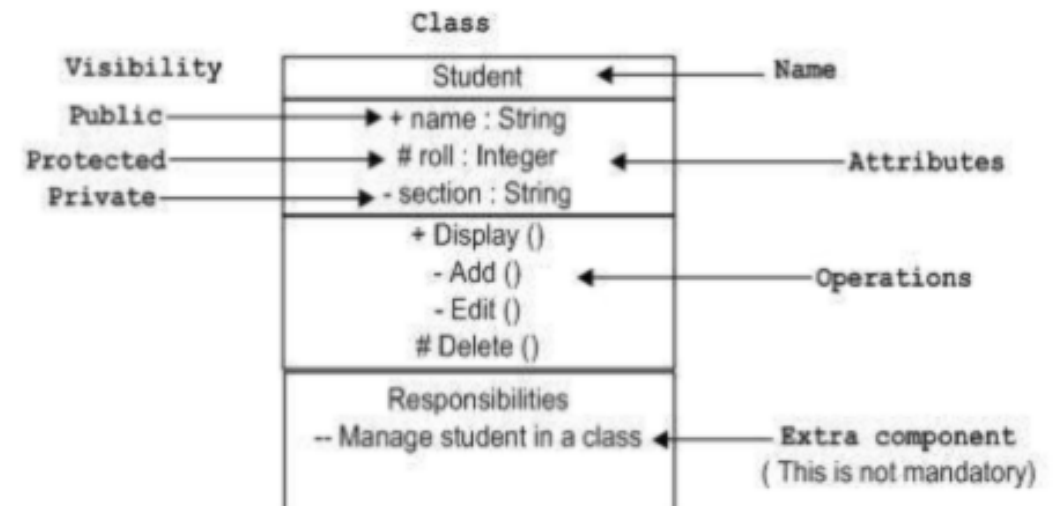
1. Structural things
2. Behavioral things
3. Grouping things
4. Annotational things

Building blocks - Structural Things

Structural things

Nouns that depicts the static behavior of a model is termed as structural things. They display the physical and conceptual components. They include class, object, interface, node, collaboration, component, and a use case.

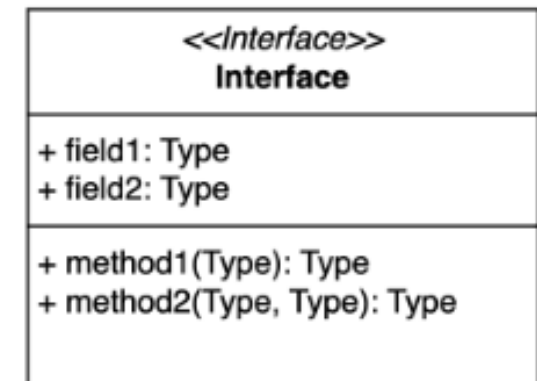
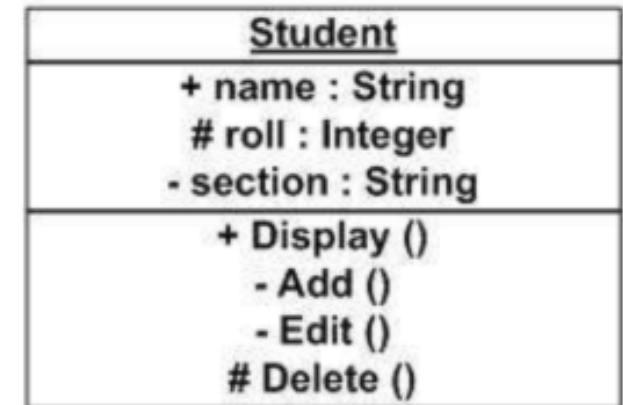
- **Class:** A Class is a set of identical things that outlines the functionality and properties of an object. It also represents the abstract class whose functionalities are not defined.



Building blocks - Structural Things

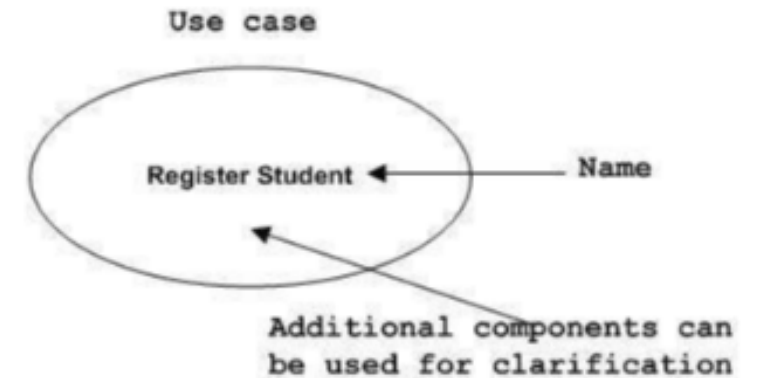
Object: An individual that describes the behavior and the functions of a system.
The notation of the object is similar to that of the class; the only difference is that the object name is always underlined

Interface: A set of operations that describes the functionality of a class, which is implemented whenever an interface is implemented.



Building blocks - Structural Things

Use case: Use case is the core concept of object-oriented modeling. It portrays a set of actions executed by a system to achieve the goal.

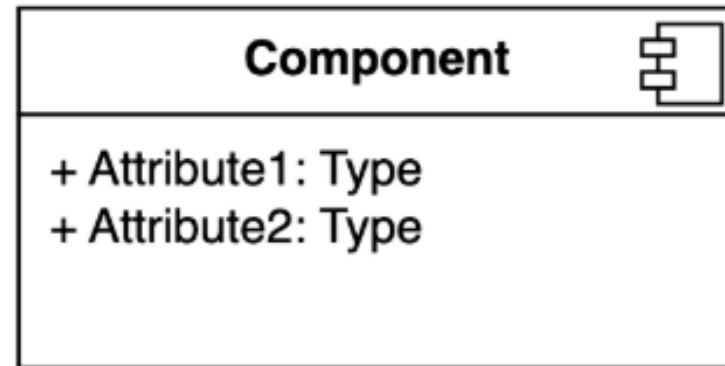


Actor: It comes under the use case diagrams. It is an object that interacts with the system, for example, a user.

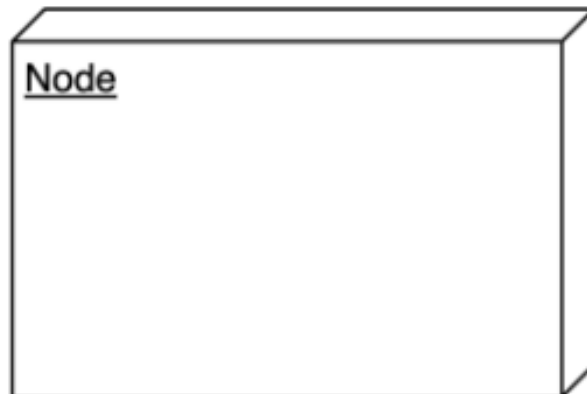


Building blocks - Structural Things

Component: It represents the physical part of the system.



Node: A physical element that exists at run time

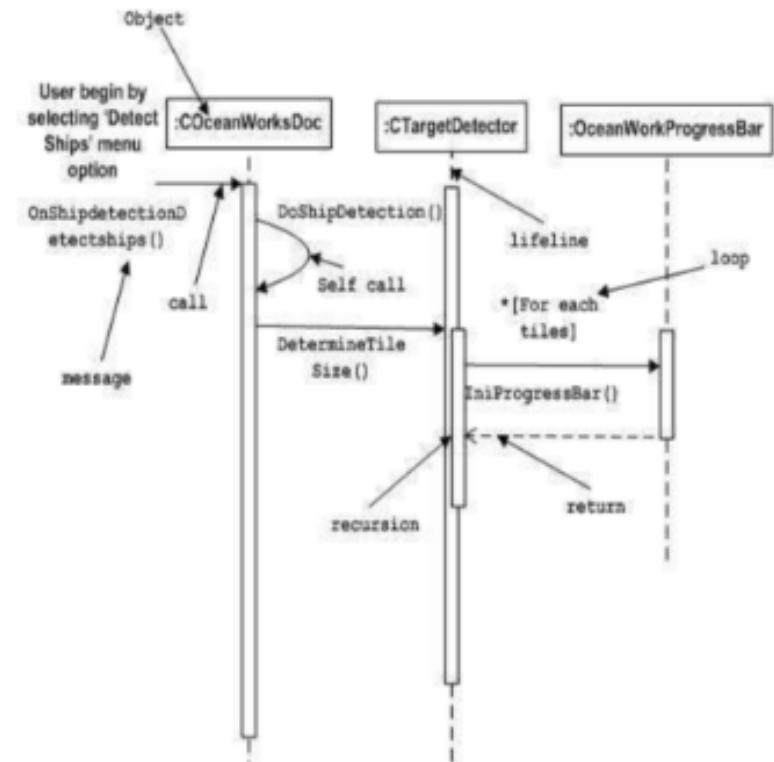
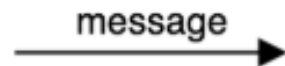


Building blocks - Behavioral Things

Behavioral Things

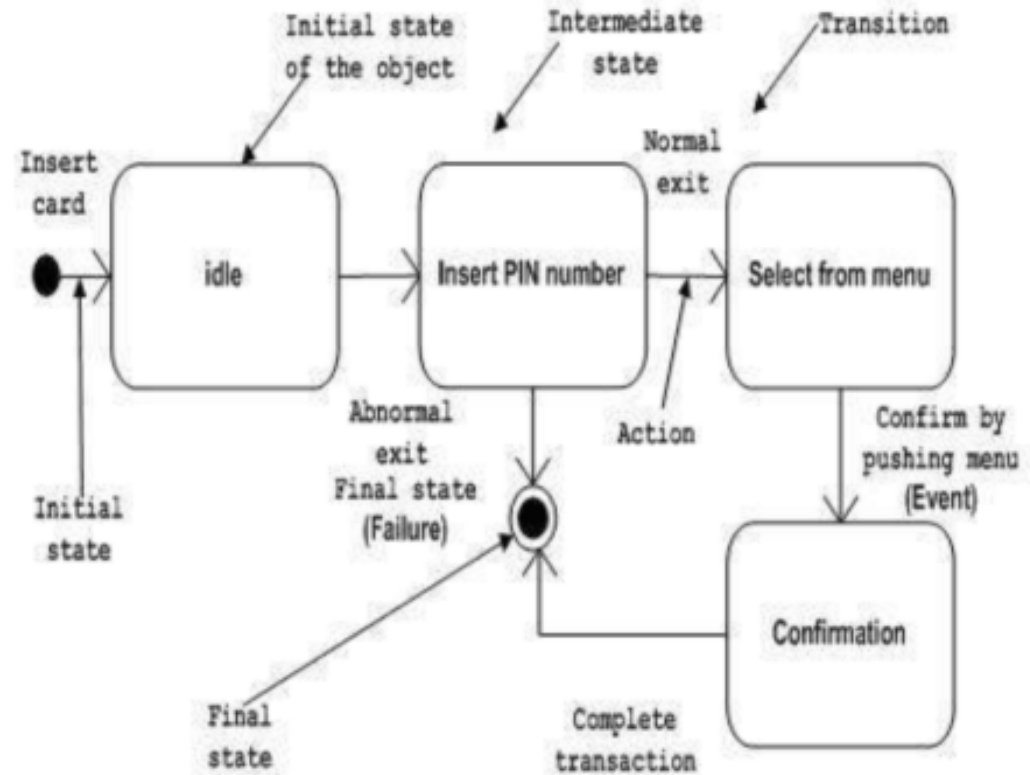
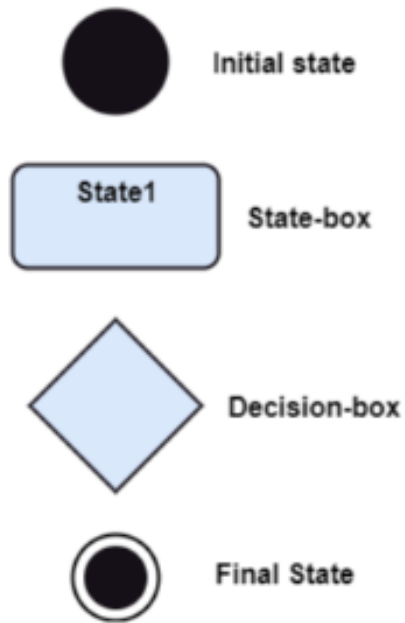
They are the verbs that encompass the dynamic parts of a model. It depicts the behavior of a system. They involve state machine, activity diagram, interaction diagram, grouping things, annotation things

- **Interaction** is defined as a behavior that consists of a group of messages exchanged among elements to accomplish a specific task.



Building blocks - Behavioral Things

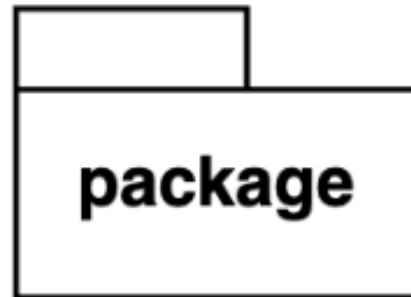
- **State machine** is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change



Building blocks - Group Things

Grouping things can be defined as a mechanism to group elements of a UML model together. There is only one grouping thing available:

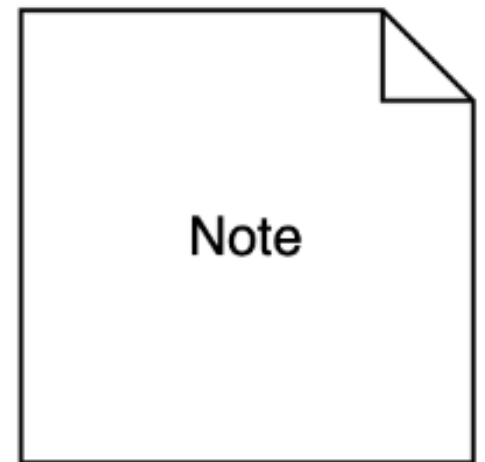
- **Package** is the only one grouping thing available for gathering structural and behavioral things.



Building blocks - Annotational Things

Annotational things can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements.

- **Note** - It is the only one Annotational thing available. A note is used to render comments, constraints, etc. of an UML element.



Building blocks - Relationship

Relationship illustrates the meaningful connections between things. It shows the association between the entities and defines the functionality of an application:

- **Dependency** is a relationship between two things in which change in one element also affects the other



- **Association** is basically a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship



- **Generalization** can be defined as a relationship which connects a specialized element with a generalized element. It basically describes the inheritance relationship in the world of objects



- **Realization** can be defined as a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other one implements them. This relationship exists in case of interfaces



Building blocks - Diagrams

Diagrams are the graphical implementation of the models that incorporate symbols and text. Each symbol has a different meaning in the context of the UML diagram. There are thirteen different types of UML diagrams that are available in UML 2.0, such that each diagram has its own set of a symbol. And each diagram manifests a different dimension, perspective, and view of the system:

Structural Diagram

It represents the static view of a system by portraying the structure of a system. It shows several objects residing in the system

1. Class diagram
2. Object diagram
3. Package diagram
4. Component diagram
5. Deployment diagram

Behavioral Diagram

It depicts the behavioral features of a system. It deals with dynamic parts of the system

2. Activity diagram
3. State machine diagram
4. Use case diagram

Interaction Diagram

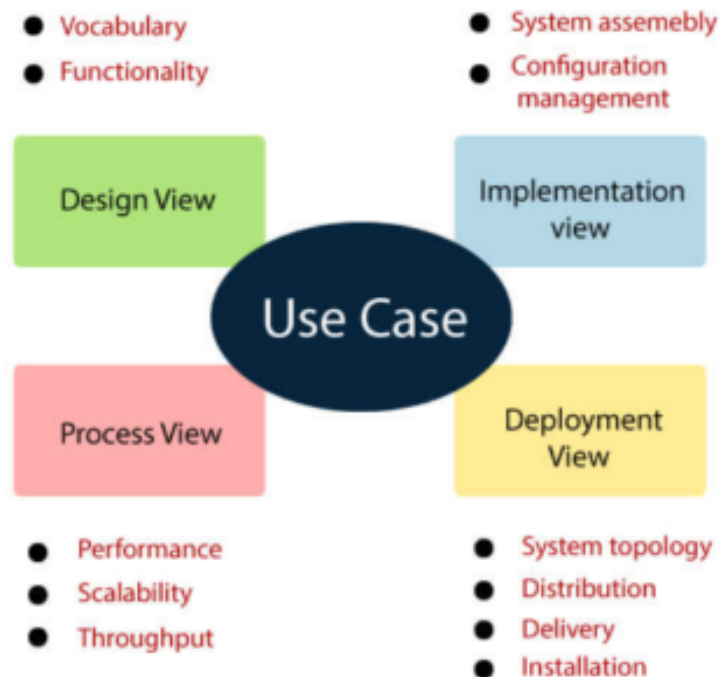
It is a subset of behavioral diagrams. It depicts the interaction between two objects and the data flow between them

3. Timing diagram
4. Sequence diagram
5. Collaboration diagram

UML - Architecture

Any real-world system is used by different users. The users can be developers, testers, business people, analysts, and many more. Before designing a system, the architecture is made with different perspectives in mind. The most important part is to visualize the system from the perspective of different viewers. These perspectives are:

- Use-case view
- Design view
- Implementation view
- Process view
- Deployment view



UML - Architecture

- The center is the **Use Case** view which connects all these four. A Use Case represents the functionality of the system. Hence, other perspectives are connected with use case.
- **Design** of a system consists of classes, interfaces, and collaboration. UML provides class diagram, object diagram to support this.
- **Implementation** defines the components assembled together to make a complete physical system. UML component diagram is used to support the implementation perspective.
- **Process** defines the flow of the system. Hence, the same elements as used in Design are also used to support this perspective.
- **Deployment** represents the physical nodes of the system that forms the hardware. UML deployment diagram is used to support this perspective.

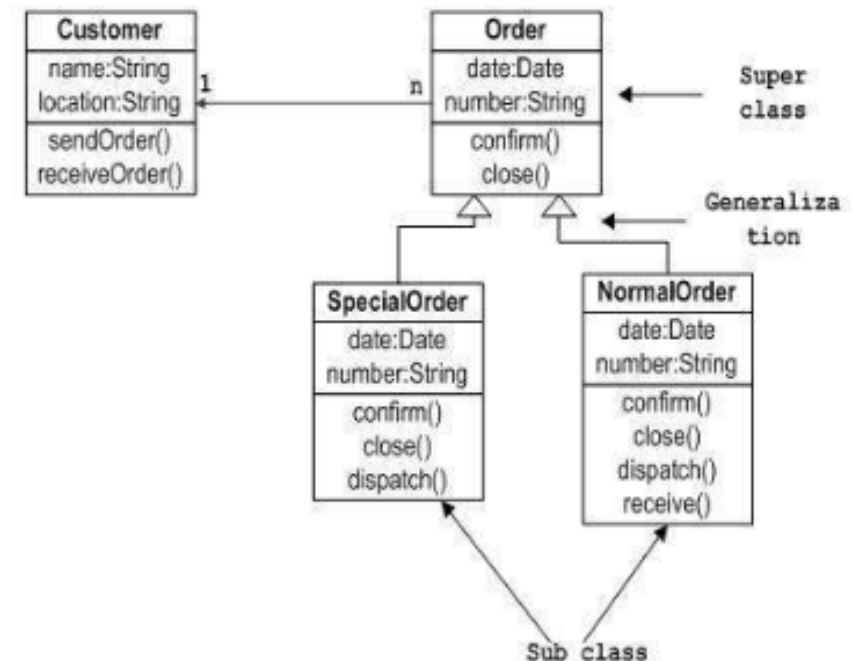
Class Diagram

- Class diagram is a static diagram and represents the static view of an application
- Class diagram describes the attributes and operations of a class
- Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints
- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.

Class Diagram

Example

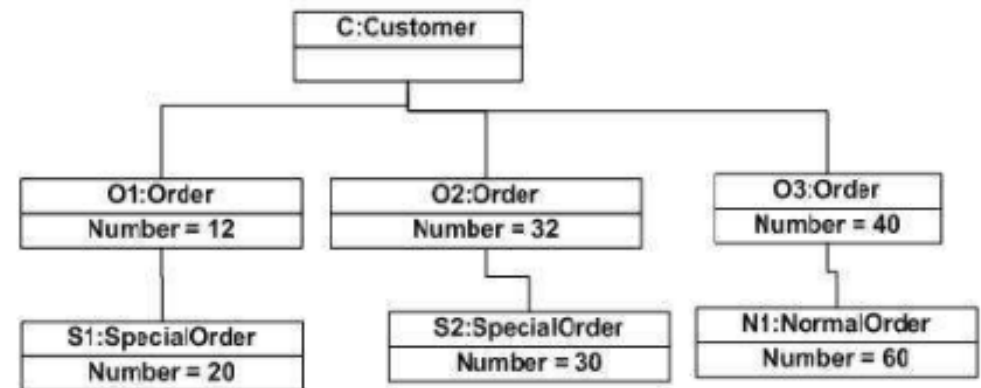
- **Order** and **Customer** are identified as the two elements of the system. They have a one-to-many relationship because a customer can have multiple orders.
- Order class is an abstract class and it has two concrete classes (inheritance relationship) **SpecialOrder** and **NormalOrder**.
- The two inherited classes have all the properties as the Order class. In addition, they have additional functions like dispatch() and receive().



Object Diagram

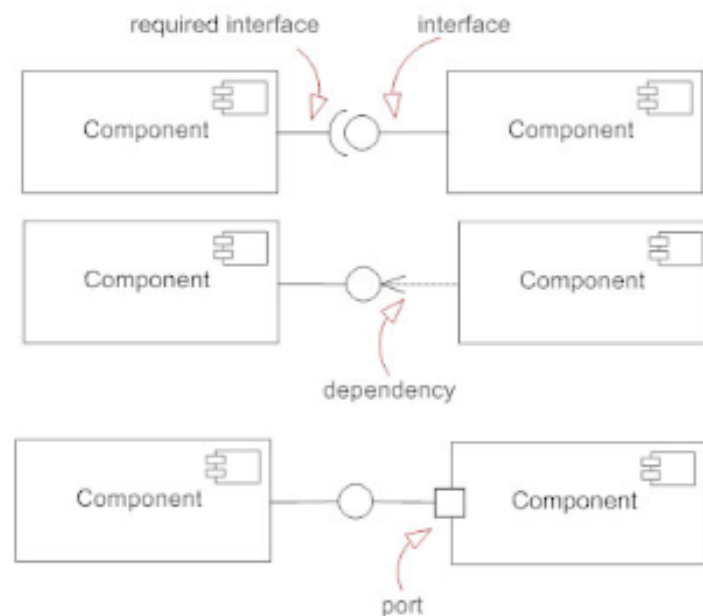
- Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams
- Object diagrams represent an instance of a class diagram. The basic concepts are similar for class diagrams and object diagrams
- Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment
- Object diagrams are used to render a set of objects and their relationships as an instance.

- A particular state which is running.
- A particular number of passengers. which will change if the snap is taken in a different time



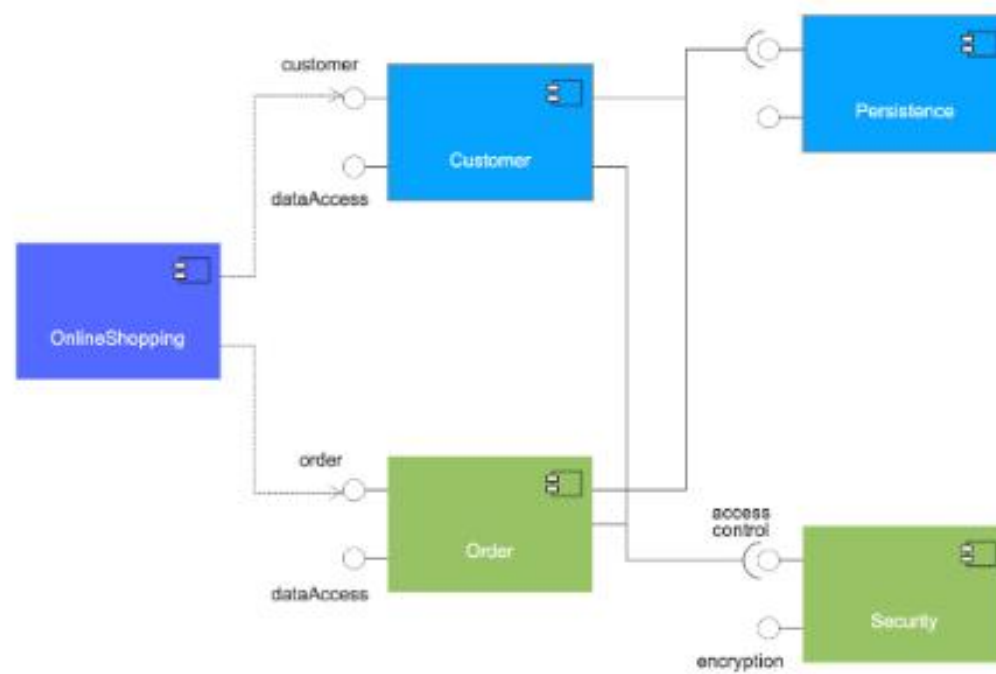
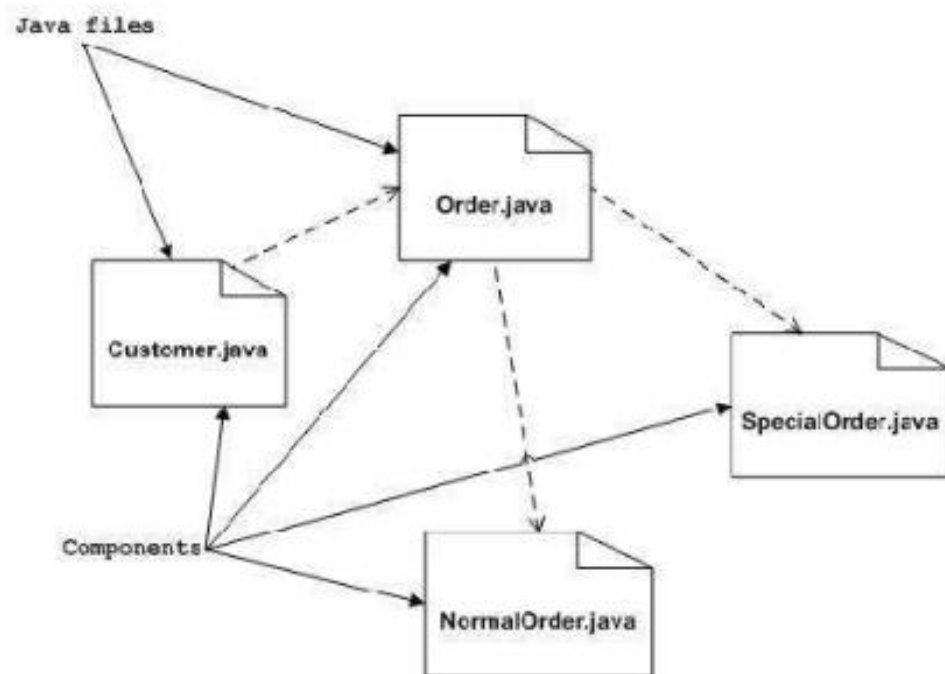
Component Diagram

- Component diagrams are used to model the physical aspects of a system. Physical aspects are the elements such as executables, libraries, files, documents, etc
- Component diagrams are used to visualize the organization and relationships among components in a system:
 - Model the components of a system.
 - Model the database schema.
 - Model the executables of an application.
 - Model the system's source code.



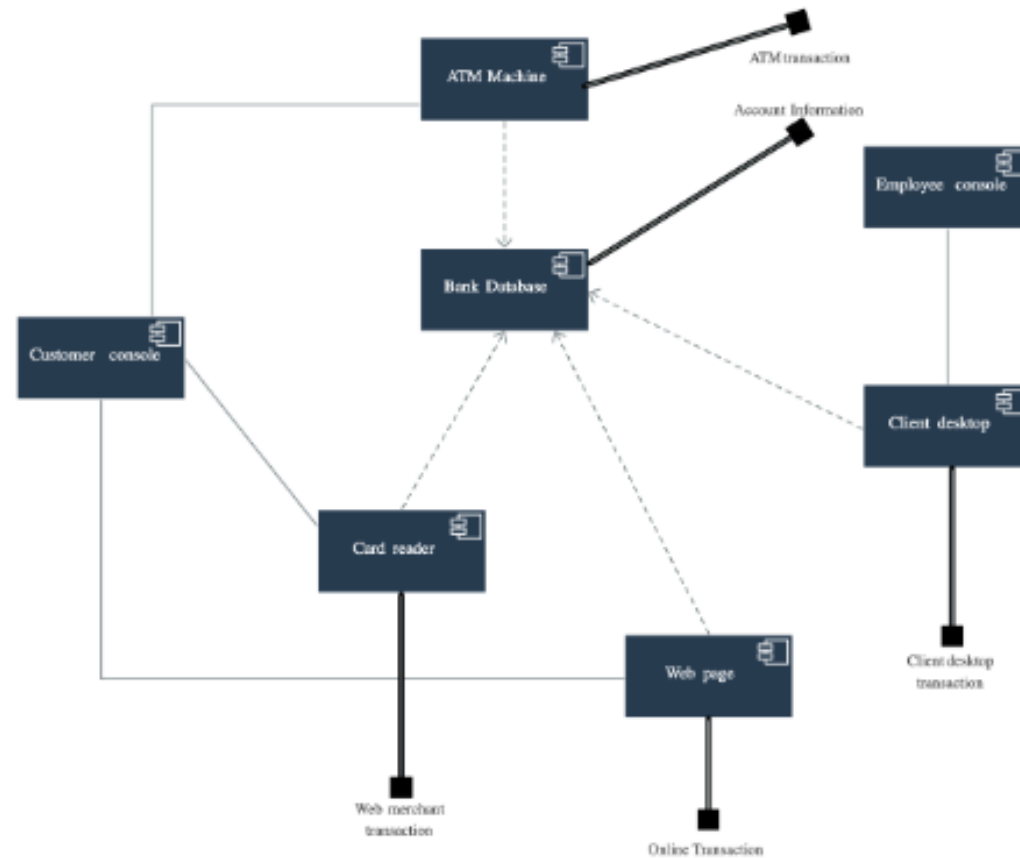
Component Diagram

Example



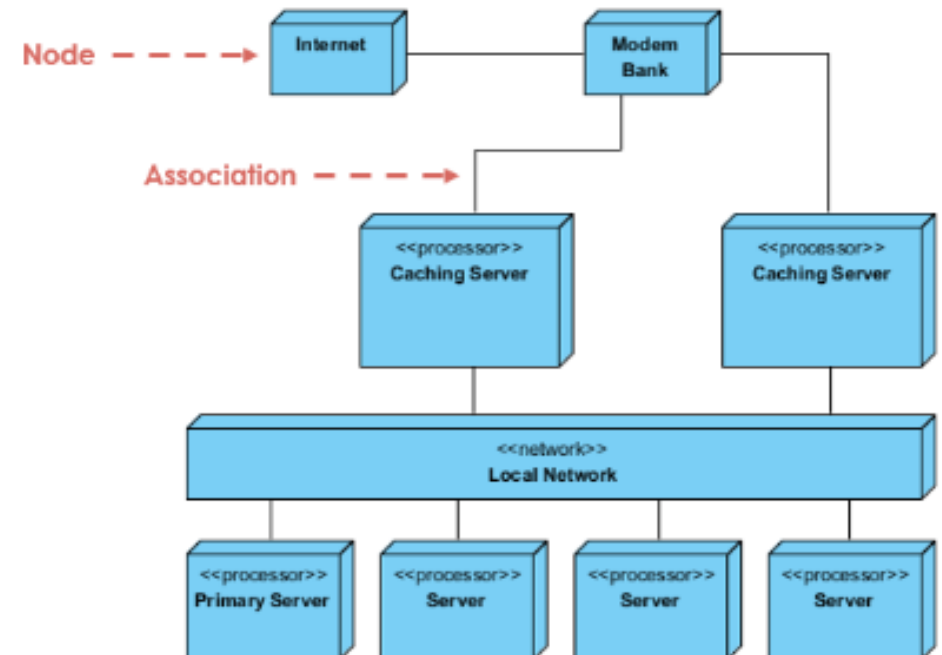
Component Diagram

ATM system



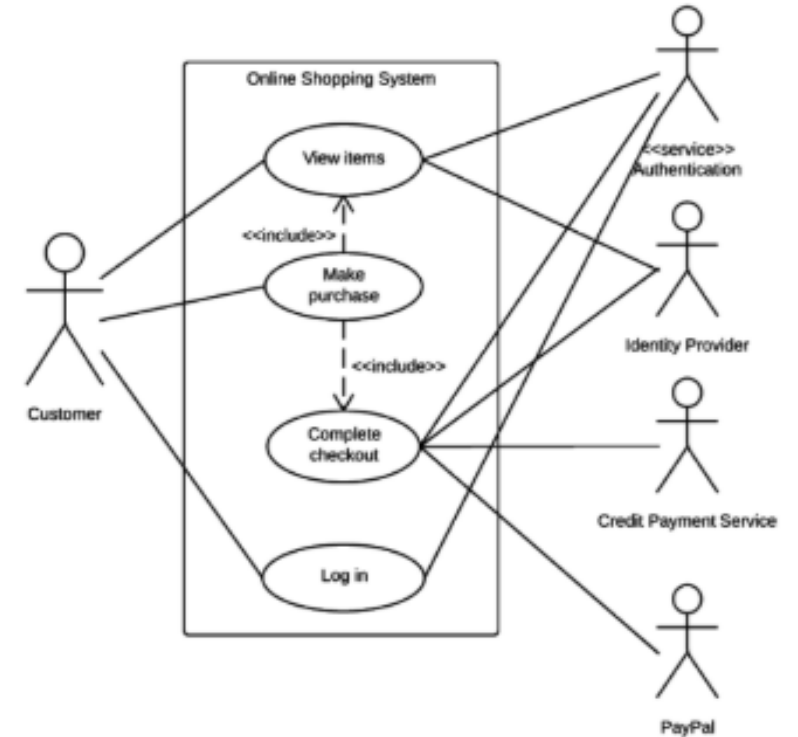
Deployment Diagram

- Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed.
- Deployment diagrams are used to describe the static deployment view of a system. Deployment diagrams consist of nodes and their relationships.
- The purpose:
 - Visualize the hardware topology of a system.
 - Describe the hardware components used to deploy software components.
 - Describe the runtime processing nodes



Use-case Diagram

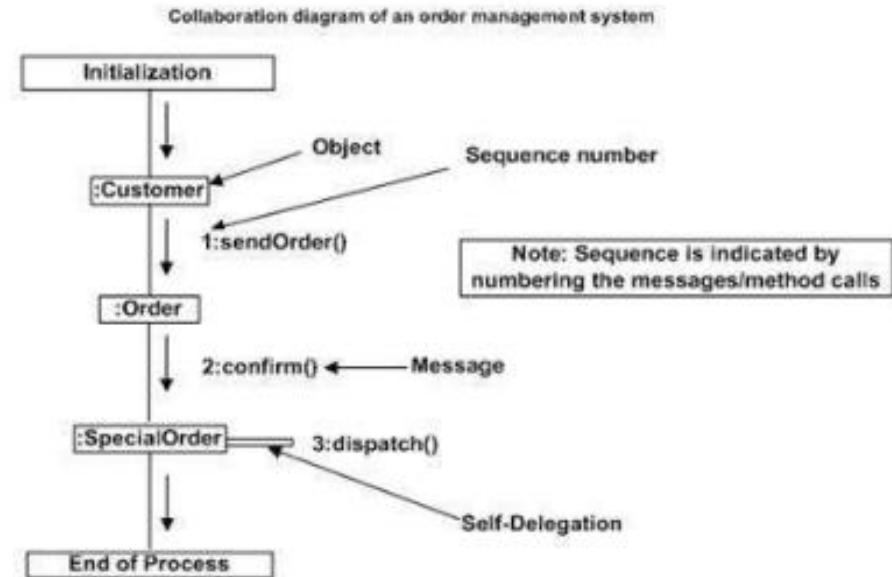
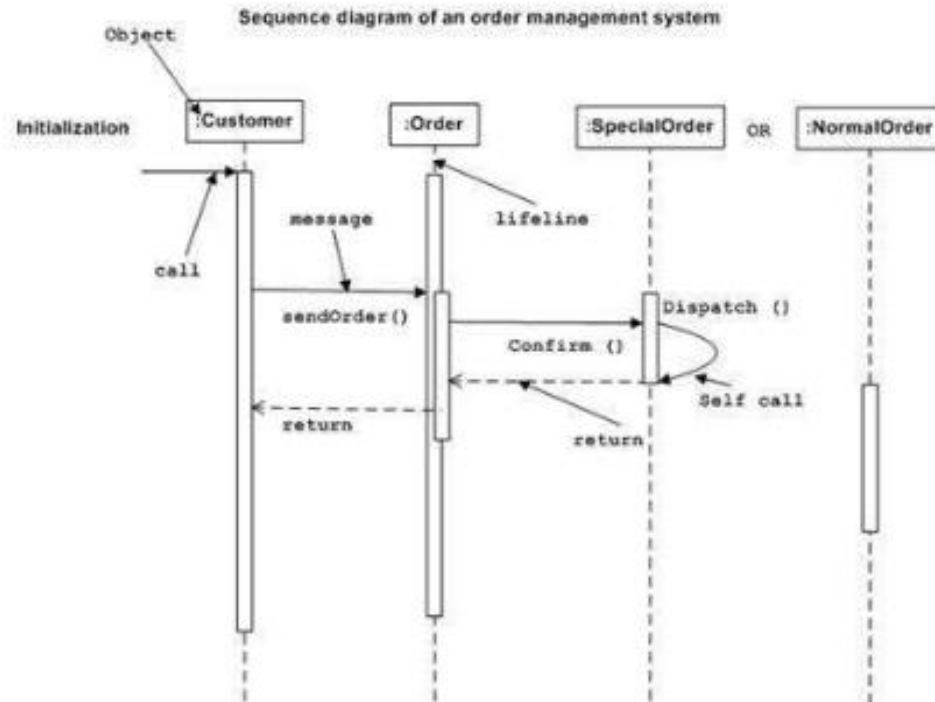
- Use-case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system
- Use case diagrams are ideal for:
 - Representing the goals of system-user interactions
 - Defining and organizing functional requirements in a system
 - Specifying the context and requirements of a system
 - Modeling the basic flow of events in a use case



Interaction Diagram

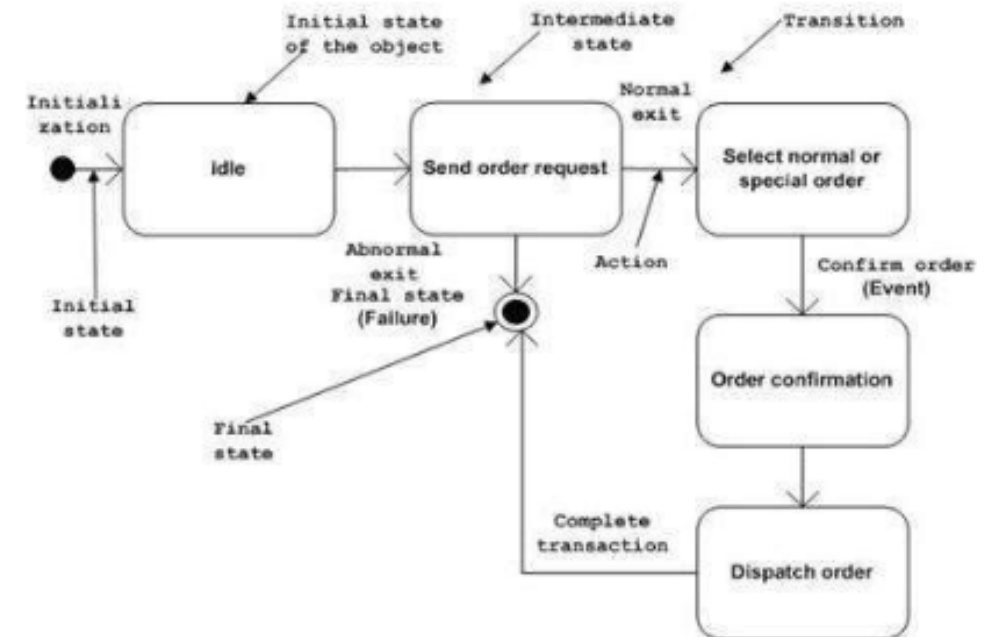
- Interaction diagram describes some type of interactions among the different elements (**Sequence & Collaboration**)
- The purpose of interaction diagram:
 - To capture the dynamic behaviour of a system.
 - To describe the message flow in the system.
 - To describe the structural organization of the objects.
 - To describe the interaction among objects.

Interaction Diagram



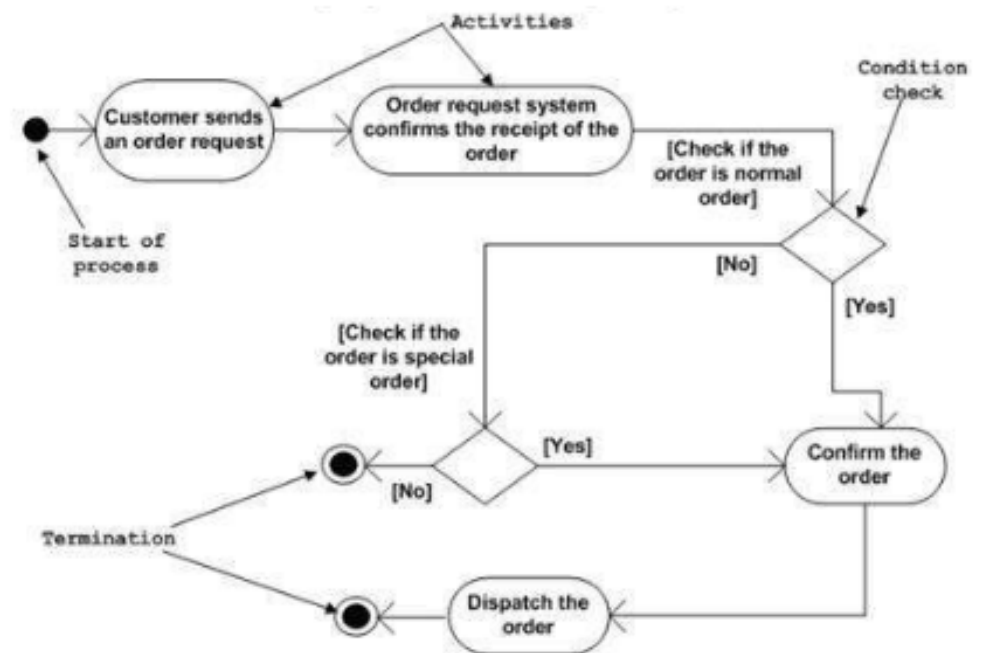
Statechart Diagram

- Statechart diagram describes different states of a component in a system. The states are specific to a component/object of a system
- A Statechart diagram describes a state machine. State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events.



Activity Diagram

- Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.
- Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.



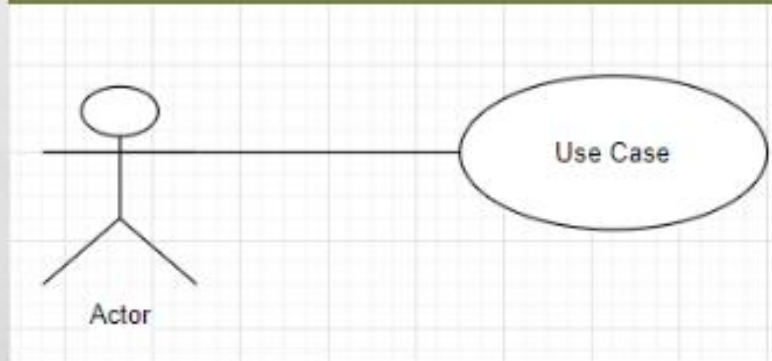
USE CASE SPECIFICATION



USE CASE SPECIFICATION

Just showing the use case diagram in UML notation is not enough. Each use case accompanied by text explaining the purpose of the use case as well as what functionality is accomplished when a use case is executed.

User case diagram



Use Case Specification

ID, Title, Description, Actors,
Preconditions, Post Conditions,
Trigger
Basic flow
Alternative flows
Exception flows.
Business rules
Non-functional requirements
Wireframe

USE CASE SPECIFICATION

- **SUMMARY**
- **THREE TYPES OF FLOWS**
- **ADDITIONAL INFORMATION**

SUMMARY

Components	Standard format/Descriptions	Example
ID	UC-NUMBER	UC-1, UC-2,UC-3,UC-n
Title	Verb + Noun Phrase	Sign up as a Publisher
Description		This use case describe how to sign up as a publisher in the system.
Actors	Someone or something that performs an action in a use case	Publisher System
Preconditions	The state that must be in place for the use case to begin	New publisher is entitled to create new publisher account. New publisher has his preferred user name, initial password, e-mail address.
Post conditions	The state that will be in place after the use case has finished	The publisher account have been setup in the system.
Trigger	The event that starts the use case.	New publisher elects to sign up an account publisher in the system.

SUMMARY

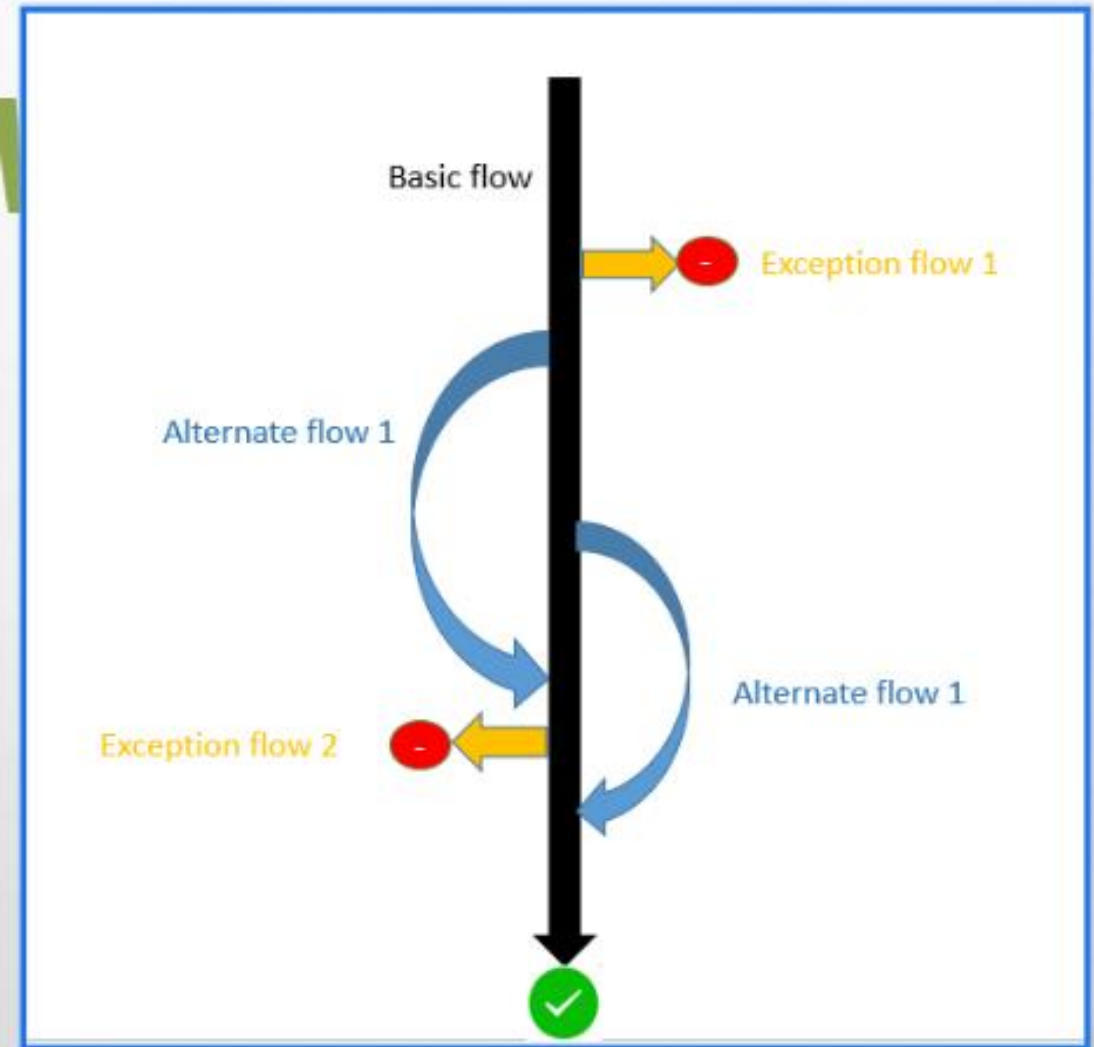
[UC-1]: Create a new Publisher

Description	This use case describe how to sign up as a publisher in the system.
Actors	New Publisher System
Preconditions	New publisher is entitled to create new publisher account. New publisher has his preferred user name, initial password, e-mail address.
Postconditions	The publisher has been set up in the system.
Trigger	New publisher elects to sign up an account publisher in the system.

THREE TYPES OF FLOW

The Process Part of the Use Case. And the process part of the use case is a collection of flows. Flows are sequences of steps.

Three types of flows are Basic Flow, Alternate flow and Exception flow.



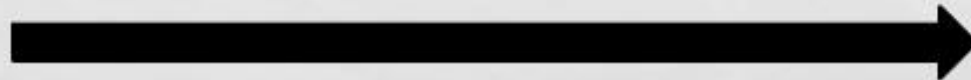
FLOW COMPONENTS

Flow Components	Descriptions
Step ID	Unique identifier for the step in the flow (e.g. BF-1,BF-2,...AF-A..)
Actor	Person or system performing the action.
Action	What is done in that step
Notes/references	Explanations, references to other entities, etc.

BASIC FLOWS

- Simplest set of steps taken by the actors to achieve the goal.
- There is only one basic flow per use case.
- It sometimes called the **happy path** because it represents the best possible path for the actors

Star use case



End use case



THE BASIC FLOW

Step ID	Actor	Action	Notes and References
BF-1	Client Service Rep	Client Service Rep logs into the system.	UC-7: Logon
BF-2	System	System displays Main Menu.	
BF-3	Client Service Rep	CSR elects to set up a new user.	
BF-4	System	System displays the New User Set-Up view.	
BF-5	Client Service Rep	CSR enters the following info: • Preferred User Name • Initial Password • E-Mail Address	

1

2

3

4

5

6

7

8

9

10

THE BASIC FLOW

Start use case

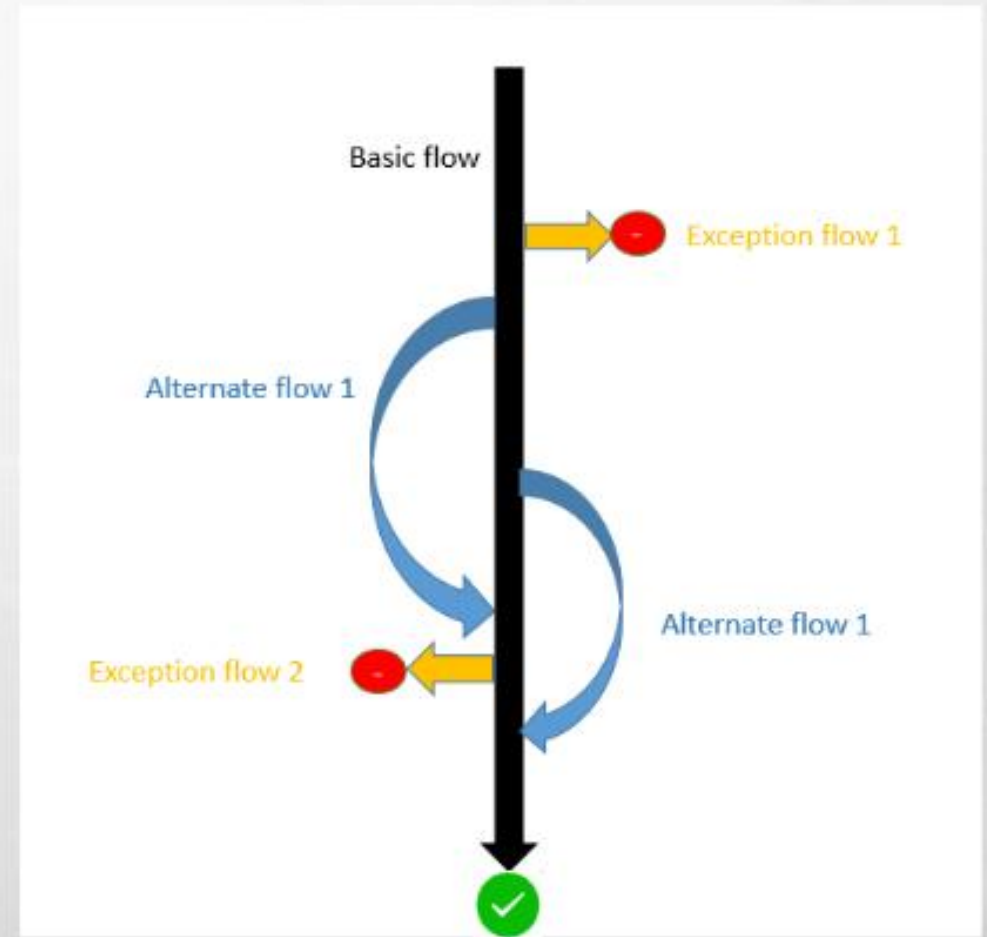


End use case

Step ID	Actor	Action	Notes and References
BF-6	System	System verifies User Name is available and in proper format.	BR-1 (User Names Unique) BR-2 (User Name Format) AF-A (User Name not Available) AF-B (User Name in Invalid Format)
BF-7	System	System verifies Password is in valid format.	BR-3 (Password Format) AF-C (Password in Invalid Format)
BF-8	System	System verifies E-Mail address is in valid format.	BR-4 (E-Mail Format) AF-D (E-Mail Address in Invalid Format)
BF-9	System	System creates user in database.	EF-A (Problem with Updating Database)
BF-10	System	System displays confirmation message: "This user has been set up in the system: • [Preferred User Name] • [Initial Password] • [E-Mail Address]"	Extension: UC-9 (Send Credentials to New User)

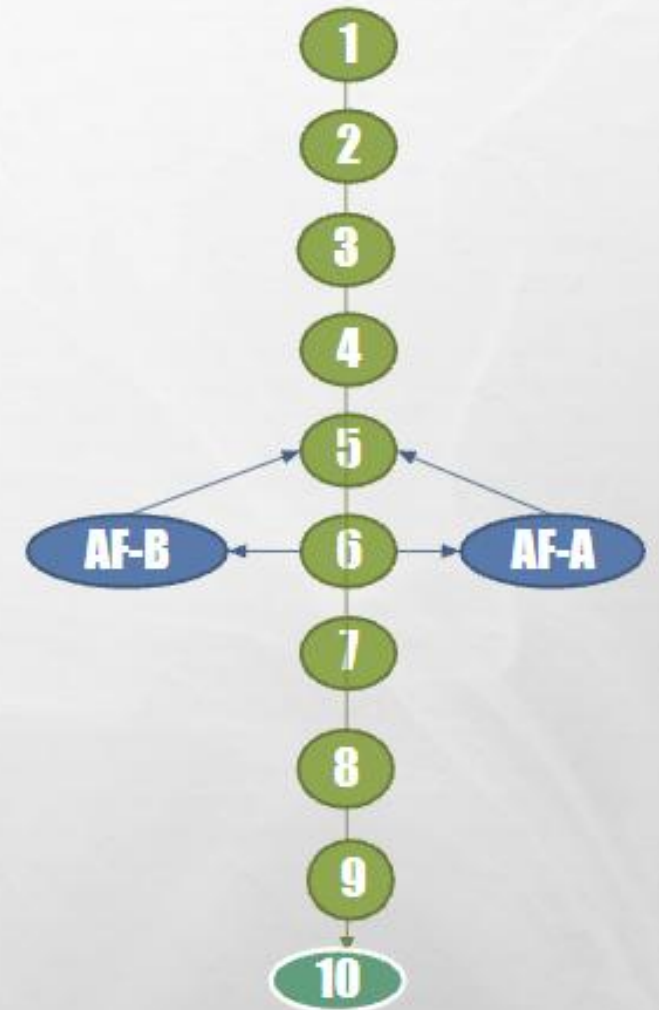
ALTERNATE FLOWS

An **alternate flow** describes a scenario other than the basic **flow** that results in a user completing his or her goal. It is often considered to be an optional **flow**. It implies that the user has chosen to take **an alternative** path through the system.



ALTERNATE FLOWS

Step ID	Actor	Action	Notes and References
BF-6	System	System verifies User Name is available and in proper format.	BR-1 (User Names Unique) BR-2 (User Name Format) AF-A (User Name not Available) AF-B (User Name in Invalid Format)



ALTERNATE FLOWS

Alternate Flows

AF-A: User Name not Available

Step ID	Actor	Action	Notes and References
AF-A-1	System	If the User Name is unavailable, the system displays the following error message to the user: "This user name is not available. Please choose another."	
AF-A-2		[Return to BF-5]	



ALTERNATE FLOWS

AF-B: User Name in Invalid Format

Step ID	Actor	Action	Notes and References
AF-B-1	System	If the User Name is in an invalid format, the system displays the following error message to the user: "This user name is invalid. User names must be 6-20 alphanumeric characters."	BR-2 (User Name Format)
AF-B-2		[Return to BF-5]	



EXCEPTION FLOWS

The goal of the use case is not met. These usually happen when something goes wrong with the system.

BF-9	System	System creates user in database.	EF-A (Problem with Updating Database)
------	--------	----------------------------------	---------------------------------------



EXCEPTION FLOWS

EF-1: Problem with Updating Database

Step ID	Actor	Action	Notes and References
EF-A-1	System	If the system encounters an unexpected issue while attempting to create a user account in the database, the system will display the following message to the user: "The system was unable to process your request due to an error in the system. Please contact the Technology Help Desk."	
EF-A-2		[End of Use Case]	



1

2

3

4

5

6

7

8

9

EF-A

10

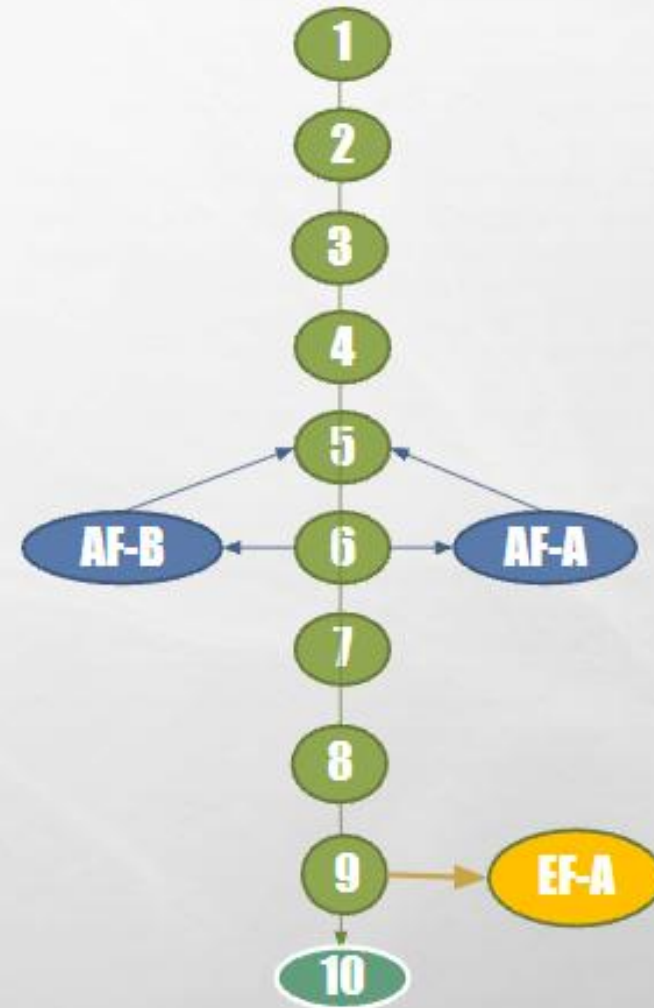
WRAP UP

1. Number of each flows in a use case:

- Basic flow: 1
- Alternate flows: 0-n
- Exception flows: 0-n

2. The result of each flows:

- Basic flow and alternate flows: meet the goal.
- Exception flows: not meet the goal.



ADDITIONAL INFORMATION

BUSINESS RULES

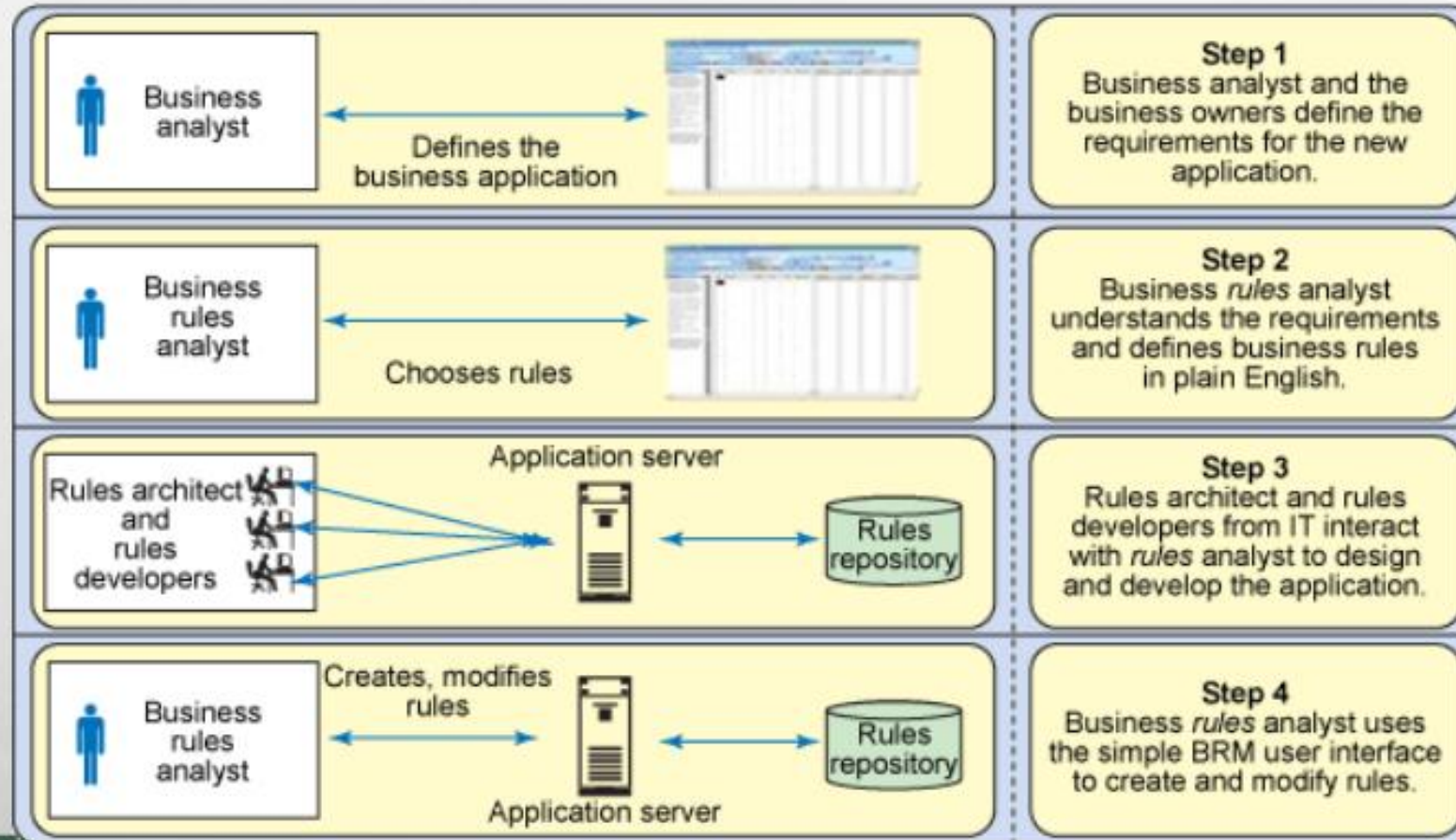
1. Business Rules are the rules that define in business.
2. They are stored in their own repository and reference them in the use cases.

Step ID	Actor	Action	Notes and References
BF-6	System	System verifies User Name is available and in proper format.	BR-1 (User Names Unique) BR-2 (User Name Format) AF-A (User Name not Available) AF-B (User Name in Invalid Format)

BUSINESS RULES

Rule ID	Name	Description
BR-1	User Names Unique	Any preferred user name may not be added to the database if it matches any existing user name.
BR-2	User Name Format	User names must adhere to the following rules: • Must be 6-20 characters long Must be alphanumeric characters only.
BR-3	Password Format	Passwords must adhere to the following rules: • Must be 6-20 characters long Must be alphanumeric characters only
BR-4	E-Mail Format	E-mails must be of format: [x]@[y].[z], where: • x is an alphanumeric string • y is an alphanumeric string • z is three or fewer characters <i>and</i> z is alphabetical characters only.

BUSINESS RULES



NON-FUNCTIONAL REQUIREMENTS:

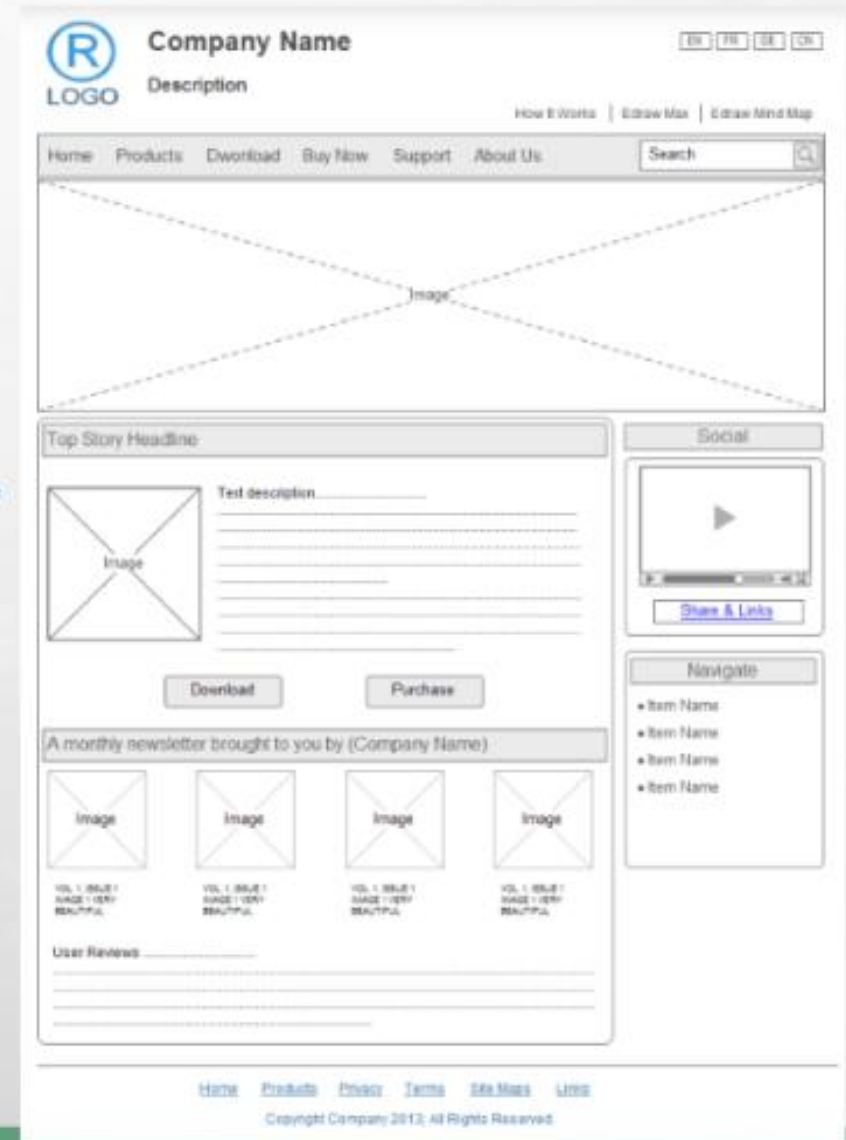
Because Use cases only display Functional Requirements, we need to add Non-function Requirements.

Non-functional Requirement

Rule ID	Name	Description
NFR-1	Time out in login sceen	Time out of login sceen is less 60 seconds.
NFR-2	User's password	User's password must be hashed by MD5.

WIREFRAMES

Wireframes are UI design illustrations.



ASSIGNMENT

- **WRITE THE USE CASE SPECIFICATION FOR YOUR PROJECT.**